

第4章 構文解析

4.1 構文解析の基礎

4.1.1 BNF 定義と CF 文法

プログラミング言語の構文は CF 文法 (文脈自由文法, context-free grammar) によって定義される。

定義 4.1. CF 文法 G は $G = \{N, \Sigma, P, S\}$ の四つ組で定義される。ここで、 N は非終端記号の集合、 Σ は終端記号の集合、 P は生成規則の集合、 $S \in N$ は開始記号である。 P の各生成規則は $A \rightarrow \alpha$ の形をしている。ここで、 $A \in N$, $\alpha \in (N \cup \Sigma)^*$ である。

プログラミング言語の構文解析における CF 文法では、 Σ は字句解析によって得られるトークンの集合とするのが一般的である。

PL2¹ のプログラムにおける手続きの定義は以下の文法で定められている。'procedure' は予約語、'IDENT' は識別子のトークン、'(', ')', ':', および ',' は区切り記号である。

$$\begin{aligned} \langle \text{proc_decl} \rangle &\rightarrow \text{'procedure'} \langle \text{proc_name} \rangle \text{'('} \text{' '}' \text{' ':'} \langle \text{inblock} \rangle \\ \langle \text{proc_decl} \rangle &\rightarrow \text{'procedure'} \langle \text{proc_name} \rangle \text{'('} \langle \text{id_list} \rangle \text{' ':'} \langle \text{inblock} \rangle \\ \langle \text{proc_name} \rangle &\rightarrow \text{'IDENT'} \\ \langle \text{id_list} \rangle &\rightarrow \text{'IDENT'} \\ \langle \text{id_list} \rangle &\rightarrow \langle \text{id_list} \rangle \text{' ,' 'IDENT'} \end{aligned}$$

$\langle \text{proc_decl} \rangle$ は識別子のトークンである名前 proc_name を持つ手続きを定義する。本体は $\langle \text{inblock} \rangle$ で記述されるプログラムで定義される。

$\beta, \gamma \in (N \cup \Sigma)^*$, $A \in N$, $A \rightarrow \alpha$ に対して、 $\beta A \gamma \Rightarrow \beta \alpha \gamma$ を P による**導出**という。導出は終端記号と非終端記号の系列に対して、生成規則の左辺の非終端記号を右辺で置き換える操作である。 $\alpha_1, \alpha_2, \dots, \alpha_n \in (N \cup \Sigma)^*$ において、 $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ のとき、 $\alpha_1 \Rightarrow^* \alpha_n$ と書くことにする。

G が定義する言語は $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$ で定義される。すなわち開始記号 S から P の生成規則を用いて導出し、すべての非終端記号がおきかえられた終端記号の系列である。

G によってプログラミング言語が定義されているとき、原始プログラムを字句解析した系列 w が $w \in L(G)$ であるとき、構文としてそのプログラムは正しいといえる。 $w \notin L(G)$ であれば、構文エラーがあると判断し、目的コードの生成は行わない。

例 4.1. 以下の PL2 プログラムは $\langle \text{proc_decl} \rangle$ から生成される系列である。ここで、 $\langle \text{inblock} \rangle \Rightarrow^* \text{'IDENT' : '=' IDENT' + 'NUMBER'}$ とする。

¹ 学生実験 c において対象とする原始プログラムを記述するプログラミング言語

```

procedure main(x,y):
    x:=y+1

```

BNF 定義

形式的にはプログラミング言語は文脈自由文法で定義されるが、文脈自由文法では生成規則の集合を記述すれば十分であることが多い。このため、生成規則の集合でプログラミング言語を定義する。その際に一般にBNF(Backus-Nauer Form) 定義と呼ばれる記法が用いられる。BNF は生成規則とほぼ同じであるが、 $\rightarrow$ の代わりに $::=$ を用いる。また、同じ左辺をもつ規則をまとめて、右辺を $|$ で並べて記載する。 $\langle \text{proc_decl} \rangle$ についてのBNF 定義は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &::= \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' ' ' ';'} \langle \text{inblock} \rangle \\
 &\quad | \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' ' ' } \langle \text{id_list} \rangle \text{' ' ';'} \langle \text{inblock} \rangle \\
 \langle \text{proc_name} \rangle &::= \text{'IDENT'} \\
 \langle \text{id_list} \rangle &::= \text{'IDENT'} \\
 &\quad | \langle \text{id_list} \rangle \text{' ' 'IDENT'}
 \end{aligned}$$

最左導出と最右導出

導出の途中に現れる系列 $\alpha \in (N \cup \Sigma)^*$ において、 α が複数の非終端記号を含むときどの非終端記号を置き換えて導出するかによって、複数の導出が存在する。例えば、 A, B を非終端記号に含み、 $\{a, b, c\} \subseteq \Sigma$, $A \rightarrow a$, $B \rightarrow b$ という生成規則を持つとき、 ABc に対する導出は $ABc \Rightarrow aBc$ と $ABc \Rightarrow Abc$ がある。 $\alpha \in (N \cup \Sigma)^*$ に対する導出のうち、最も左にある非終端記号を置き換える導出を**最左導出**、最も右にある非終端記号を置き換える導出を**最右導出**といい、それぞれ $\alpha \Rightarrow_{lm} \beta$, $\alpha \Rightarrow_{rm} \beta$ のように記述する。

α の $A \rightarrow \gamma$ による最左導出 $\alpha \Rightarrow_{lm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_1 \in \Sigma^*$ である。すなわち、 α の A より左の α_1 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

α の $A \rightarrow \gamma$ による最右導出 $\alpha \Rightarrow_{rm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_2 \in \Sigma^*$ である。すなわち、 α の A より右の α_2 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

最左導出、最右導出の0回以上の繰り返しをそれぞれ $\Rightarrow_{lm}^*$, $\Rightarrow_{rm}^*$ で表す。

例 4.2. $PL2$ の $\langle \text{proc_decl} \rangle$ からの最左導出は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &\Rightarrow_{lm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{' ' ';'} \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' } \text{'IDENT'} \text{'(' } \langle \text{id_list} \rangle \text{' ' ';'} \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' } \text{'IDENT'} \text{'(' } \langle \text{id_list} \rangle \text{' ' 'IDENT' ' ' ';'} \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure' } \text{'IDENT'} \text{'(' 'IDENT' ' ' 'IDENT' ' ' ';'} \langle \text{inblock} \rangle
 \end{aligned}$$

さらに $\langle \text{inblock} \rangle \Rightarrow_{lm}^* \text{'IDENT' ':=' 'IDENT' ' ' '+' 'NUMBER'}$ であるとき

$$\Rightarrow_{lm}^* \text{'procedure' } \text{'IDENT'} \text{'(' 'IDENT' ' ' 'IDENT' ' ' ';'} \text{'IDENT' ':=' 'IDENT' ' ' '+' 'NUMBER'}$$

$PL2$ の $\langle \text{proc_decl} \rangle$ からの最右導出は以下のようになる。

$\langle \text{proc_decl} \rangle \Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$

ここで $\langle \text{inblock} \rangle \Rightarrow_{rm}^* \text{'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$ であるとき

$\Rightarrow_{rm}^* \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{' ,' 'IDENT' ') ' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

開始記号 S からの 0 回以上の導出によって得られる $\alpha \in (N \cup \Sigma)^*$ (ここで, $S \Rightarrow^* \alpha$) のことを **文形式** (sentential form) という. さらに, $S \Rightarrow_{lm}^* \beta$, $S \Rightarrow_{rm}^a st\gamma$ となる $\beta, \gamma \in (N \cup \Sigma)^*$ のことをそれぞれ左文形式, 右文形式という.

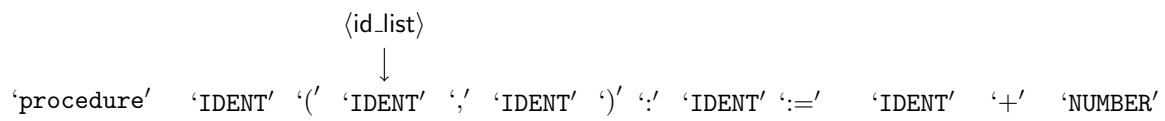
4.1.2 解析木

導出の過程は木構造で表現することができる. 非終端記号, 終端記号を頂点として表し, $A \rightarrow \alpha$ で導出したとき, A の頂点から $\alpha = \alpha_1\alpha_2\cdots\alpha_n$ ($\alpha_i \in N \cup \Sigma$) の α_i に有向辺を追加する. S からの導出をこのようにして表すと, S の頂点を根, $a \in \Sigma$ の頂点を葉とする木構造が構成される. この木構造のことを**解析木** (parse tree)(あるいは**構文木** (syntax tree)) という.

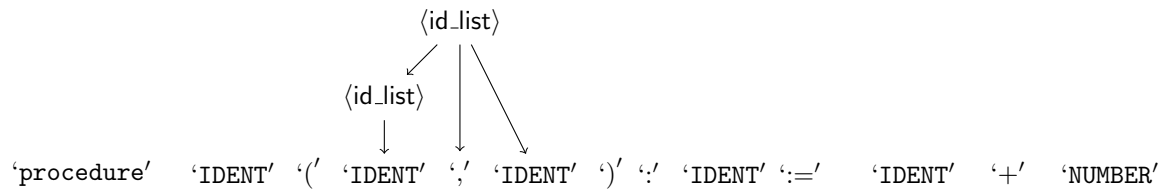
例 4.1 の例の (最左) 導出は以下ようになる.

$\langle \text{proc_decl} \rangle \Rightarrow \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' ,' 'IDENT' ') ' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' } \langle \text{var_decl_part} \rangle \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' } \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' } \langle \text{assignment_statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= } \langle \text{expression} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= } \langle \text{expression} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= } \langle \text{term} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= } \langle \text{factor} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= } \langle \text{var_name} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{factor} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{number} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT' ') ' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

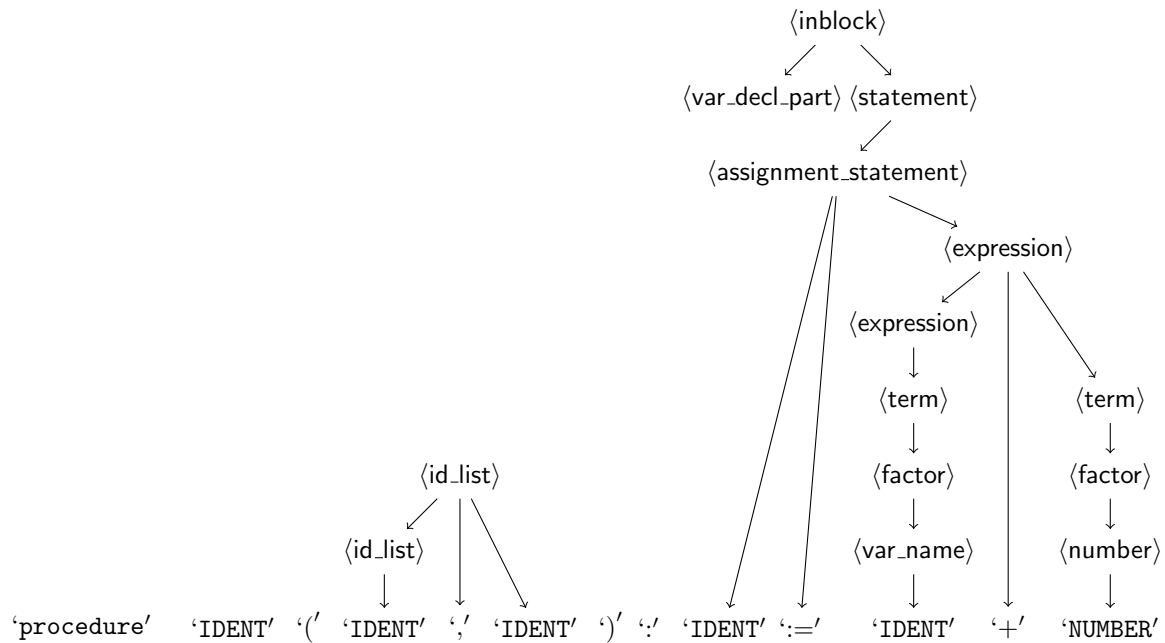
この導出から構成される木構造は以下ようになる.



次に, $\langle \text{id_list} \rangle \rightarrow \langle \text{id_list} \rangle \text{'IDENT'}$ を逆向きに適用する.



以下, 右辺にマッチする生成規則を適用し, $\langle \text{inblock} \rangle$ の部分木を生成する.



最後に $\langle \text{proc_decl} \rangle$ を左辺に持つ規則によって木が構成される.

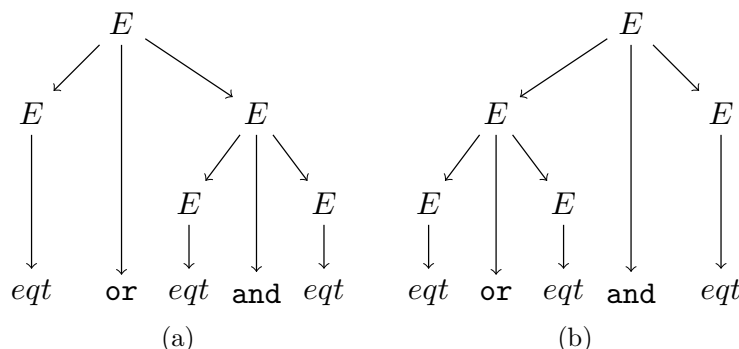


図 4.1: 複数の解析木

$L(G_1)$ に属する $eqt\ or\ eqt\ and\ eqt$ という系列に対して図 4.1 のような 2 つの解析木が存在する。導出において、 $\underline{E}$ を置き換えの対象となった E とすると、(a) は $\underline{E} \Rightarrow \underline{E}\ or\ E \Rightarrow eqt\ or\ \underline{E} \Rightarrow eqt\ or\ \underline{E}\ and\ E \Rightarrow eqt\ or\ eqt\ and\ \underline{E} \Rightarrow eqt\ or\ eqt\ and\ eqt$, (b) は $\underline{E} \Rightarrow \underline{E}\ and\ E \Rightarrow \underline{E}\ or\ E\ and\ E \Rightarrow eqt\ or\ \underline{E}\ and\ E \Rightarrow eqt\ or\ eqt\ and\ \underline{E} \Rightarrow eqt\ or\ eqt\ and\ eqt$, という導出に対する解析木となる。

このように 1 つのトークン系列に対して 2 つ以上の異なる解析木が存在するとき文法は曖昧であるという。

3 つの eqt のトークンが具体的にそれぞれ $x = 1, y = 2, z = 3$ であるとする。 x, y, z がすべて 1 であるとする、 $x = 1$ は真、 $y = 2, z = 3$ は偽となる。 and, or がそれぞれ論理積、論理和を表すとする、(a) の解釈では、最初の $eqt(x = 1)$ が真となるので、全体の論理式は真であるが、(b) の解釈では最後の $eqt(z = 3)$ が偽であるので論理式は偽となる。

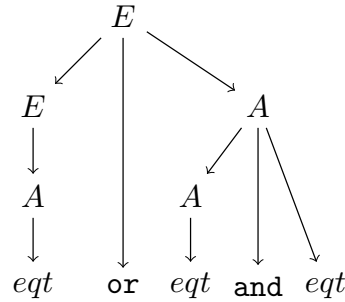
曖昧な文法では入力トークンの系列の意味が定まらないため、プログラミング言語の文法は曖昧でないことが望まれる。

論理和と論理積の場合、一般には和が積が優先されると解釈されるため、(a) の解釈をとる。これにそって G_1 の曖昧さを除去した文法 G_2 を以下に示す。

$$G_2 = \langle \{E, A\}, \{or, and, eqt\}, P_2, E \rangle$$

$$\text{ここで, } P_2 = \left\{ \begin{array}{l} E \rightarrow E\ or\ A \\ E \rightarrow A \\ A \rightarrow A\ and\ eqt \\ A \rightarrow eqt \end{array} \right\}$$

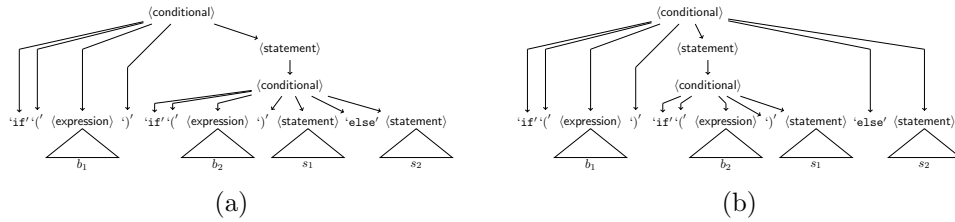
$L(G_1) = L(G_2)$ であり、 G_2 は曖昧ではない。これは、 and を or より優先するために、新たな非終端記号 A を導入して、 and が解析木で or よりも葉に近いように解釈されるようにしたためである。 G_2 では、単一生成規則 (single production) “ $E \rightarrow A$ ” を導入したため解析木のサイズが増大する。 n レベルの優先度の導入のために n 個の非終端記号が導入され、 n 個の単一生成規則が必要になる。曖昧性を除去するトレードオフとして、解析木が大きくなり、解析により長い時間がかかることになる。



このため、生成規則の適用順序を決めて解析の曖昧性を除去する場合もある。C言語などの条件分岐は `else` 付きの場合と `if ~ then ~ else ~`, `else` なしの場合 `if ~ then ~` の場合がある。

$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' else' } \langle \text{statement} \rangle \end{aligned}$$

b_1, b_2 を $\langle \text{expression} \rangle$ から導出されるトークン列, s_1, s_2 を $\langle \text{statement} \rangle$ から導出されるトークン列とする。if (b_1) if (b_2) s_1 else s_2 に対して図??のような2つの解析木が存在する。



C言語では、`else` は最も近い条件に対するものであると解釈するので、解析木として (b) であると決め、曖昧さを除去している。したがって、 b_1 が真で、 b_2 が偽であると評価された場合に実行される。構文解析における曖昧性は望ましいものとは言えないので、プログラム言語を定義する場合は無曖昧な CF 文法で定義することを基本とする。

上記の場合は、新たな予約語 `fi` を導入することで、曖昧さを解消できる。

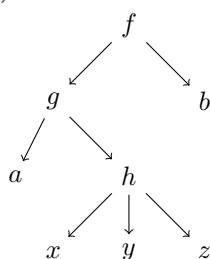
$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' fi' } \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{' else' } \langle \text{statement} \rangle \text{' fi' } \end{aligned}$$

4.1.4 解析木の表現

木の頂点の出力次数がわかっているとき解析木をトラバースする方法に基づいて、木構造を1次元的に表現することができる。木 T の根頂点を $root(T)$ と書くことにする。 $root(T)$ の次数が n であるとき $root(T)$ からの子を根頂点とする木を左から $T_1, T_2, \dots, T_n$ とする。

- **プリオーダー** $root(T)$ の次に T_1 のプリオーダー系列 $pre(T_1)$ を連結し、その次に $pre(T_2)$ を連結し、というのを再帰的に T_n まで繰り返す.
- **インオーダー** T_1 のインオーダーの系列 $in(T_1)$ に $root(T)$ を連結し、その次に $in(T_2), \dots, in(T_n)$ を連結する.
- **ポストオーダー** T_1 のポストオーダー $post(T_1), \dots, post(T_n)$ を連結し、最後に $root(T)$ を連結する.

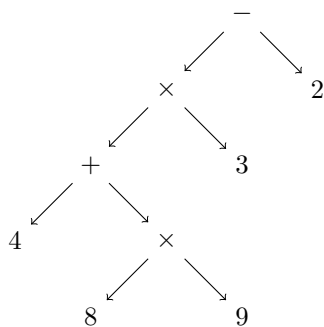
関数を表す式 $f(g(a, h(x, y, z)), b)$ に対して、関数名とその引数を親子関係として木構造で表すと、以下ようになる.



この木構造はプリオーダーで表すと $fgahxyzb$ となる.

f が 2 引数, g が 2 引数, h が 3 引数であることが既知であれば、この系列から関数の形を再現できる.

四則演算の算術式 $(4 + 8 \times 9) \times 3 - 2$ は以下のような構造を持つ.



これをポストオーダー系列で表すと $489 \times +3 \times 2-$ となる.

この記法は**逆ポーランド記法**と呼ばれる. それぞれの演算の引数が決まっていればスタックに対するルールに従って計算すれば式の評価が行われる.

逆ポーランド記法における評価の方法は以下のとおりである.

1. 数字がきたらスタックに積む
2. 演算がきたら引数の数だけスタックからポップして演算を施し、その結果をスタックにプッシュする.
3. 入力を全部読んだあとにスタックトップのあるのが評価値

$489 \times +3 \times 2-$ をスタックと対にしてこの方法で評価すると以下ようになる.

$$\begin{aligned}
(489 \times +3 \times 2-, []) &\rightarrow (89 \times +3 \times 2-, [4]) \\
&\rightarrow (9 \times +3 \times 2-, [4, 8]) \\
&\rightarrow (\times +3 \times 2-, [4, 8, 9]) \\
&\rightarrow (+3 \times 2-, [4, 72]) \\
&\rightarrow (3 \times 2-, [76]) \\
&\rightarrow (\times 2-, [76, 3]) \\
&\rightarrow (2-, [228]) \\
&\rightarrow (-, [228, 2]) \\
&\rightarrow (\epsilon, [226])
\end{aligned}$$

プリオーダーは、木の根を開始記号とする最左導出の規則適用に相当し、ポストオーダーは、木の根を開始記号とする最右導出の逆順の規則適用に相当する。

4.1.5 First と Follow

入力系列 w が与えられたとき、できるだけ効率的に CF 文法に従って $w \in L(G)$ (あるいは $w \notin L(G)$) を示したい。すなわち、 $S \Rightarrow^* w$ となる導出を示す必要がある。この際に導出に使う生成規則を選択する必要がある。例えば、 $A \rightarrow aB, A \rightarrow bC$ という2つの規則があり、 $\beta A \gamma$ という文形式から次の導出を行う。このとき、両方の生成規則が適用可能である。簡単のために最左導出であると考え、 $\beta \in \Sigma^*$ であり、入力文字列 w の接頭語が βa であるならば $A \rightarrow aB$ を選択すべきであり、接頭語が βb であるならば、 $A \rightarrow bC$ を選択すべきである。

このように次に処理すべきトークン (上の場合は β の次のトークン) によって生成規則の選択を決めることができる場合がある。

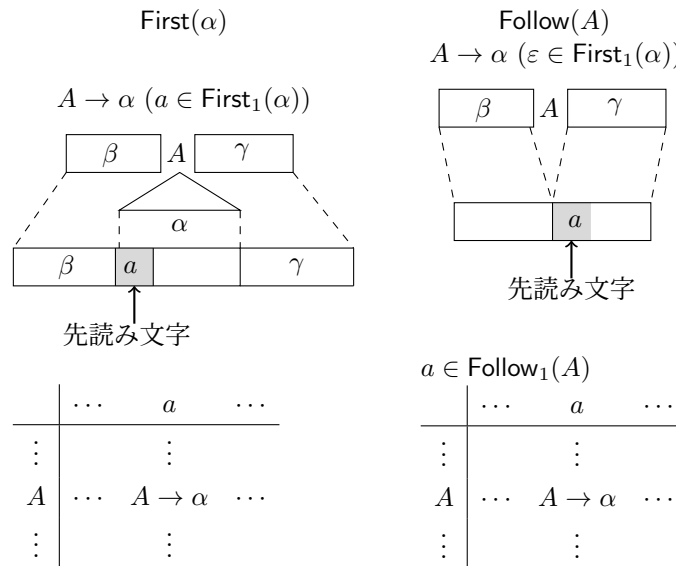


図 4.2: 導出の際の先読みによる決定

さらに、 $A \rightarrow \varepsilon$ の場合は、 A に続く文字が何かによってこの生成規則を適用できるかどうかを決めることができる場合がある。

コンパイラの構文解析においては、このようなまだ読み込んでいない**先読み文字**の情報をを用いて、生成規則の選択を決めることが行われる。もしこのような先読み文字の情報をを用いない場合は、すべての選択を試してみることになる。生成規則の選択がまちがっていた場合は、まちがいをやりなおす**バックトラック**によって規則を選び直さないといけなくなる。 βb が接頭語であるにもかかわらず、 $A \rightarrow aB$ で書き換えた場合は、書き換えをやりなおして、 $A \rightarrow bC$ に書き換える。このとき一旦読んでしまったトークンをまた入力系列に戻す。このような操作には時間がかかるため、一般的なプログラミング言語の設計においては生成規則の解析と先読み文字によって、適用できる生成規則が一意的になるような CF 文法で設計される。

生成規則が適用されるときに生成される接頭語を First、生成規則が適用されて生成される文字列の直後の接頭語を Follow といい、これらは生成規則をあらかじめ解析することによって得られる。

定義 4.2. CF 文法 $G = \langle N, \Sigma, P, S \rangle$ が与えられたとき、系列 $\alpha \in (N \cup \Sigma)^*$ に対する Follow_1 集合を以下のように定義する。

$$\alpha \Rightarrow^* \varepsilon \text{ のとき, } \text{First}_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\} \cup \{\varepsilon\}$$

$$\alpha \not\Rightarrow^* \varepsilon \text{ のとき, } \text{First}_1(\alpha) = \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta, \beta \in (N \cup \Sigma)^*\}$$

$\text{First}_1(\alpha)$ は、 α から導出によって生成される記号列で先頭に出現する可能性のある終端記号の集合を表す。 α から ε が生成される、つまり、 α が導出によって消失しうるときには、 $\text{First}_1(\alpha)$ には ε という特別な要素を含めることにする。明らかに $\text{First}_1(\varepsilon) = \{\varepsilon\}$ である。

$\alpha \in (N \cup \Sigma)^+$ に対して、 α の最初の記号を $\text{fst}(\alpha)$ 、最初の記号を除いた系列を $\text{fol}(\alpha)$ と書くことにする。 $(\text{fst}(\alpha) \in N \cup \Sigma, \text{fol}(\alpha) \in (N \cup \Sigma)^* \text{ となる。})$ CF 文法 $\langle N, \Sigma, P, S \rangle$ において、 PO_P を以下のように定める。 $PO_P = \{\gamma \in (N \cup \Sigma)^* \mid \beta\gamma = \alpha, \beta \in (N \cup \Sigma)^*, A \rightarrow \alpha \in P\}$ PO_P は、生成規則 $A \rightarrow \alpha$ の右辺 α の接尾語の集合を表す。

$\alpha \in PO_P \cup N$ に対して $\text{First}_1(\alpha)$ は、以下のようにして計算することができる。

アルゴリズム 4.1. (First_1 を求めるアルゴリズム):

ステップ 1: $\text{First}_1(\varepsilon) \leftarrow \{\varepsilon\}$

ステップ 2: $\alpha \in PO_P \setminus \{\varepsilon\}$ に対して

(a) $\text{fst}(\alpha) \in \Sigma$ ならば、 $\text{First}_1(\alpha) \leftarrow \{\text{fst}(\alpha)\}$

(b) $\text{fst}(\alpha) \in N$ ならば、 $\text{First}_1(\alpha) \leftarrow \emptyset$

ステップ 3: すべての $\text{First}_1(\alpha)$ ($\alpha \in PO_P \cup N$) に新たな要素が追加されなくなるまで以下を繰り返す。

(a) $\text{fst}(\alpha) = A \in N$ である α に対して $\text{First}_1(\alpha)$ に以下の要素を追加する。

$\varepsilon \notin \text{First}_1(A)$ のとき、 $\text{First}_1(A)$

$\varepsilon \in \text{First}_1(A)$ のとき、 $\text{First}_1(A) \setminus \{\varepsilon\} \cup \text{First}_1(\text{fol}(\alpha))$

(b) $A \rightarrow \beta \in P$ に対して $\text{First}_1(\beta)$

定義 4.3. CF 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して, $A \in N$ に対する $\text{Follow}_1(A)$ は以下のように定義される.

$$\begin{aligned} \text{Follow}_1(A) = & \{a \in \Sigma \mid S \Rightarrow^* \beta A a \gamma, \beta, \gamma \in (N \cup \Sigma)^*\} \\ & \cup \{\# \mid S \Rightarrow^* \beta A, \beta \in (N \cup \Sigma)^*\} \end{aligned}$$

ここで, $\#$ は入力の終わりを示す記号である.

アルゴリズム 4.2.

ステップ 1: $\text{Follow}_1(S) \leftarrow \{\#\}$

ステップ 2: $A \in N \setminus \{S\}$ に対して $\text{Follow}_1(A) \leftarrow \emptyset$

ステップ 3: $B \rightarrow \beta C \gamma \in P$ を満たす $B, C \in N$, $\beta, \gamma \in (N \cup \Sigma)^*$ に対してすべての $\text{Follow}_1(A) (A \in N)$ に新たな追加がなくなるまで以下を追加する.

(a) $\varepsilon \notin \text{First}_1(\gamma)$ のとき, $\text{First}_1(\gamma)$

(b) $\varepsilon \in \text{First}_1(\gamma)$ のとき, $\text{First}_1(\gamma) \setminus \{\varepsilon\} \cup \text{Follow}_1(B)$

4.2 下降型構文解析

4.2.1 左再帰性

手続き宣言の BNF の一部を以下に再掲する.

$$\begin{aligned} \langle \text{proc_decl} \rangle &::= \text{'procedure' } \langle \text{IDENT} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle \\ \langle \text{id_list} \rangle &::= \langle \text{id_list} \rangle \text{' , ' } \langle \text{IDENT} \rangle \\ &\quad | \langle \text{IDENT} \rangle \end{aligned}$$

$\langle \text{proc_decl} \rangle$ からの最左導出の第一段階は $\langle \text{proc_decl} \rangle \Rightarrow \text{'procedure' } \langle \text{IDENT} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$ であり, 次に $\langle \text{id_list} \rangle$ に対して導出が適用される. ここで, $\langle \text{id_list} \rangle ::= \langle \text{id_list} \rangle \text{' , ' } \langle \text{IDENT} \rangle$ は左再帰的 (left-recursive) と呼ばれる. 右辺の最初に左辺と同じ非終端記号を含むので

$$\begin{aligned} \langle \text{id_list} \rangle &\Rightarrow_{lm} \langle \text{id_list} \rangle \text{' , ' } \langle \text{IDENT} \rangle \\ &\Rightarrow_{lm} \langle \text{id_list} \rangle \text{' , ' } \langle \text{IDENT} \rangle \text{' , ' } \langle \text{IDENT} \rangle \\ &\Rightarrow_{lm} \langle \text{id_list} \rangle \text{' , ' } \langle \text{IDENT} \rangle \text{' , ' } \langle \text{IDENT} \rangle \text{' , ' } \langle \text{IDENT} \rangle \\ &\quad \vdots \end{aligned}$$

のように同じルールが際限なく適用されて $\langle \text{id_list} \rangle$ が消えない可能性がある. $\langle \text{id_list} \rangle \rightarrow \langle \text{IDENT} \rangle$ を適用すればこの導出を止めることができるが, $\langle \text{IDENT} \rangle \in \text{First}_1(\langle \text{id_list} \rangle)$ であるので, 文形式の左端によっては別の規則を選択することができない.

左再帰性除去

$\{A \rightarrow A\alpha, A \rightarrow \beta\}$ という左再帰の規則を含む生成規則に対して、以下の生成規則集合は A から同じ文形式への導出を持ち、左再帰性を持つ規則を含まない。

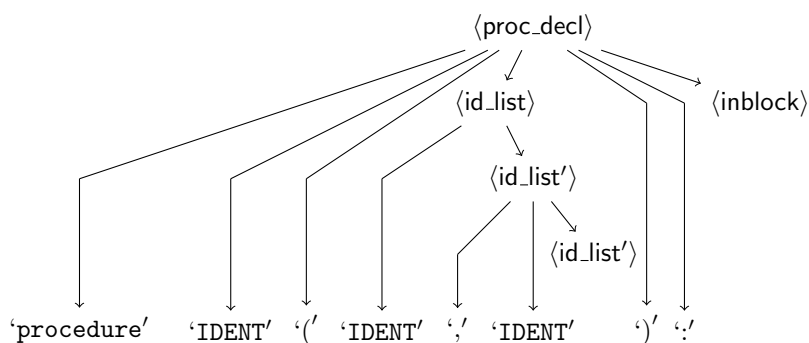
$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \\ A' &\rightarrow \varepsilon \end{aligned}$$

A' は新たな非終端記号である。

これに従って $\langle \text{id_list} \rangle$ に対して左再帰性を除去すると以下のようになる。

$$\begin{aligned} \langle \text{id_list} \rangle &::= \text{'IDENT' } \langle \text{id_list}' \rangle \\ \langle \text{id_list}' \rangle &::= \text{' , 'IDENT' } \langle \text{id_list}' \rangle \\ &\quad | \quad \varepsilon \end{aligned}$$

この場合の解析木は以下のようになる。



例 4.3. 加算 $+$ と乗算 $*$ を持つ式に対して、左再帰性をもつ生成規則と E からの導出が等しい左再帰性を除去した生成規則は以下のようになる。

$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{'IDENT' } \end{aligned}$	$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{'IDENT' } \end{aligned}$
(a) 左再帰性を持つ生成規則	(b) 左再帰性を除去した生成規則

4.2.2 左括り出し

以下の BNF 定義において

$$\begin{aligned} \langle \text{conditional} \rangle &::= \text{'if' } \langle \text{expression} \rangle \text{'then' } \langle \text{statement} \rangle \text{'else' } \langle \text{statement} \rangle \text{'fi' } \\ &\quad | \quad \text{'if' } \langle \text{expression} \rangle \text{'then' } \langle \text{statement} \rangle \text{'fi' } \end{aligned}$$

予約語 ‘if’ を見ただけではどちらの規則を適用すればよいかわからない。さらに新たな非終端記号 $\langle \text{conditional} \rangle$ を導入し、

$$\begin{aligned} \langle \text{conditional} \rangle &::= \text{‘if’} \langle \text{expression} \rangle \text{‘then’} \langle \text{statement} \rangle \langle \text{conditional} \rangle \\ \langle \text{conditional} \rangle &::= \text{‘else’} \langle \text{statement} \rangle \text{‘fi’} \\ &\quad | \text{‘fi’} \end{aligned}$$

とすれば、 $\langle \text{conditional} \rangle$ の規則は単一になり、 $\langle \text{conditional} \rangle$ についても先読みトークンが ‘else’ か ‘fi’ かで決定的に規則を選択できる。

同じ左辺を持つ生成規則において、右辺の共通な左端をまとめることを**左括り出し** (left-factoring) という。一般には $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ に対して、新たな非終端記号 A' を導入して $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$ に変換することである。

4.2.3 LL 構文解析

必要に応じて左再帰性の除去、左括りだしを施した後、最左導出によって構文解析を行う。入力系列を先頭から読みながら最左導出を適用する構文解析は **LL (Left-to-right leftmost) 構文解析** と呼ばれる。

下向き構文解析表

最左導出において、開始記号から導出される文形式で最左に非終端記号 A を含む文形式は $\beta A \gamma$ の形であり、 $\beta \in \Sigma^*$, $\gamma \in (N \cup \Sigma)^*$ である。CF 文法 G の生成規則のうち、左辺に A を持つ生成規則が以下のようにになっているとする。

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$$

入力系列 w に対する解析木を構成する場合、 $w = \beta\beta'$ であるとき $A \rightarrow \alpha_i$ による $\beta A \gamma$ の導出が w に到達するためには、以下の条件が成り立つ必要がある。

- $\beta' \neq \varepsilon$ かつ $\varepsilon \notin \text{First}_1(\alpha_i)$ のとき $\text{fst}(\beta') \in \text{First}_1(\alpha_i)$
- $\beta' \neq \varepsilon$ かつ $\varepsilon \in \text{First}_1(\alpha_i)$ のとき、 $\text{fst}(\beta') \in \text{Follow}_1(A)$
- $\beta' = \varepsilon$ のとき、 $\alpha_i = \varepsilon$ かつ $\# \in \text{Follow}_1(A)$

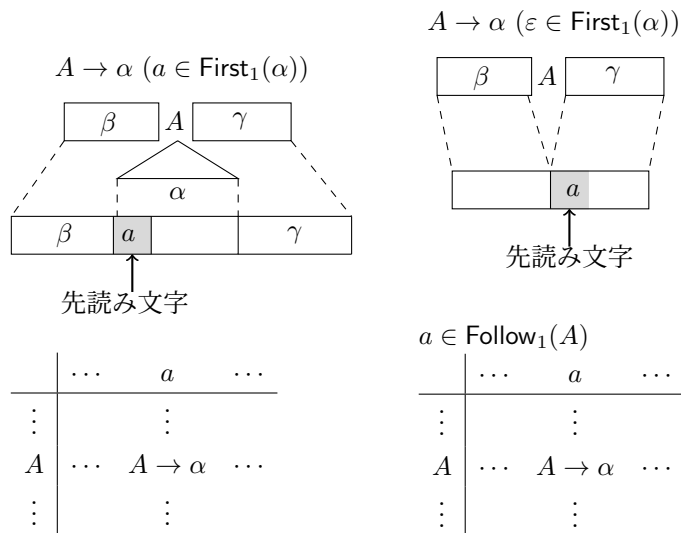
最左導出に対する下向き構文解析表は、縦向きに非終端記号ごとの行、横向きに $\Sigma \cup \{\#\}$ ごとの列を持つ表であり、以下のように構成される。

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n$$

であるとき、

- $a \in \text{First}_1(\alpha_i) \cap \Sigma$ ならば、 A の行、 a の列に $A \rightarrow \alpha_i$ を追加する。

- $\varepsilon \in \text{First}_1(\alpha_i)$ ならば、 A の行、 $a \in \text{Follow}_1(A)$ の列に $A \rightarrow \alpha_i$ を追加



LL(1) 構文解析

定義 4.4. CF 文法 G に対する下向き構文解析表において、表のエントリの生成規則が高々 1 つであるとき、 G は **LL(1) 構文解析可能**である。このとき、 G は **LL(1) 文法**である。

下向き構文解析表は、最左導出可能なすべての生成規則を数え上げる。したがって、CF 文法 G が LL(1) 構文解析可能であるとき、 $w \in L(G)$ であるとき、かつ、そのときに限り、開始記号から w への決定的な生成規則の適用によって導出が存在する。明らかに下向き構文解析表以外の生成規則を適用しても導出を構成することはできない。

LL 構文解析は、構文木を最左導出で根方向から葉方向に構成する。先読み文字（トークン）以外の入力文字を直接参照することなく、適用可能な生成規則を正確に予測しながら構成するため**予測的構文解析**とも呼ばれる。

先読み文字を k 文字読んで導出の生成規則を決定できる CF 文法を LL(k) 文法という。2 文字以上先読みすると一般にはより適用する生成規則を限定することができる。しかし、一般的には 2 以上の k が使われることは少ない。

4.2.4 拡張 CF 文法と ELL(1) 文法

拡張 CF 文法は、生成規則の右辺に正規表現の選択と Kleene 閉包を導入して拡張した CF 文法である。この拡張によって CF 文法の表現力が増加することはない。この拡張によってプログラミング言語をよりコンパクトに定義することができるようになる。

アルゴリズム 4.3. 拡張 CF 文法 $G = \langle N, \Sigma, P, S \rangle$ に対して等価な CF 文法 $G' = \langle N', \Sigma', P', S' \rangle$ を以下のようにして構成できる。 P のすべての右辺が $(N \cup \Sigma)^*$ の要素になるまでステップ 1 を繰り返す。

ステップ 1: $A \rightarrow e \in P$ のうち $e \notin (N \cup \Sigma)^*$ である生成規則を P から取り除く.

(a) $e = e_1 | e_2$ の場合: P に $A \rightarrow e_1$ と $A \rightarrow e_2$ に加える.

(b) $e = e_1 e_2$ の場合: 新たな非終端記号 B を導入し, $A \rightarrow e_1 B$ と $B \rightarrow e_2$ を P に加え, N に B を追加する.

(c) $e = e_1^*$ の場合: 新たな非終端記号 B を導入し, $A \rightarrow B$, $B \rightarrow e_1 B$, $B \rightarrow \varepsilon$ を P に追加し, N に B を追加する.

最終的に得られた N を N' とし, P を P' とする.

変換に基づいて得られる CF 文法に対して $\text{First}_1(\alpha)$ を求めることができる. この First_1 を用いて拡張 CF 文法に対する First_1 を構成できる.

定義 4.5. 拡張 CF 文法の生成規則 $A \rightarrow e$ に対して

- $e = \varepsilon$ のとき, $\text{First}_1(e) = \{\varepsilon\}$
- $e \in \Sigma$ のとき, $\text{First}_1(e) = \{e\}$
- $e = e_1 | e_2$ のとき, $\text{First}_1(e) = \text{First}_1(e_1) \cup \text{First}_1(e_2)$
- $e = e_1 e_2$ のとき,
 - $\varepsilon \in \text{First}_1(e_1)$ のとき, $\text{First}_1(e) = \text{First}_1(e_1) \setminus \{\varepsilon\} \cup \text{First}_1(e_2)$
 - $\varepsilon \notin \text{First}_1(e_1)$ のとき, $\text{First}_1(e) = \text{First}_1(e_1)$
- $e = e_1^*$ のとき, $\{\varepsilon\} \cup \text{First}_1(e_1)$

定義 4.6. $A \rightarrow e \in P$ のとき, $\text{Follow}_1(e) = \text{Follow}_1(A)$

$e = e_1 | e_2$ のとき, $\text{Follow}_1(e_1) = \text{Follow}_1(e)$, $\text{Follow}_1(e_2) = \text{Follow}_1(e)$

$e = e_1 e_2$ のとき, $\text{Follow}_1(e_2) = \text{Follow}_1(e)$

$\varepsilon \in \text{First}_1(e_2)$ のとき, $\text{Follow}_1(e_1) = \text{First}_1(e_2) \setminus \{\varepsilon\} \cup \text{Follow}_1(e)$

$\varepsilon \notin \text{First}_1(e_2)$ のとき, $\text{Follow}_1(e_1) = \text{First}_1(e_2)$

$e = e_1^*$ のとき, $\text{Follow}_1(e_1) = \text{First}_1(e_1) \setminus \{\varepsilon\} \cup \text{Follow}_1(e)$

Follow₁ は大きな項からその部分項へ計算される.

拡張 CF 文法に対しても CF 文法と同様に下向き構文解析表を構成することができる.

定義 4.7. 拡張 CF 文法 G に対する下向き構文解析表を構成したとき, 表の各エントリーの生成規則が高々 1 つであるとき, $ELL(1)$ 構文解析可能である. このとき, G は $ELL(1)$ 文法である.

4.2.5 JJTree による降下型構文解析プログラムの生成

JJTree は、JavaCC のプリプロセッサである。jjt ファイルには、生成規則と字句解析の仕様を記述する。

JJTree のプログラムには、字句解析に用いるトークンの定義と生成規則を表すクラスメソッドを記述する。例えば、 $E \rightarrow T(+T)^*$ といった生成規則は以下のようなクラスメソッドとして記述する。

```
void E() :
{
{
    T()
    ( <ADDOP> T() ) *
}
```

ここで、<ADDOP> は+のトークンである。生成規則の左辺をクラスメソッドとし、本体を右辺として記述する。JJTree は、下降型構文解析プログラムを生成する。

JJTree によるプログラム生成の流れを図??に示す。jjt ファイルを JJTree プロセッサに入力し、javacc ファイル(.jj ファイル) といくつかの java ファイルが生成される。さらに javacc で変換し、すべてのファイルを java ファイルに変換する。

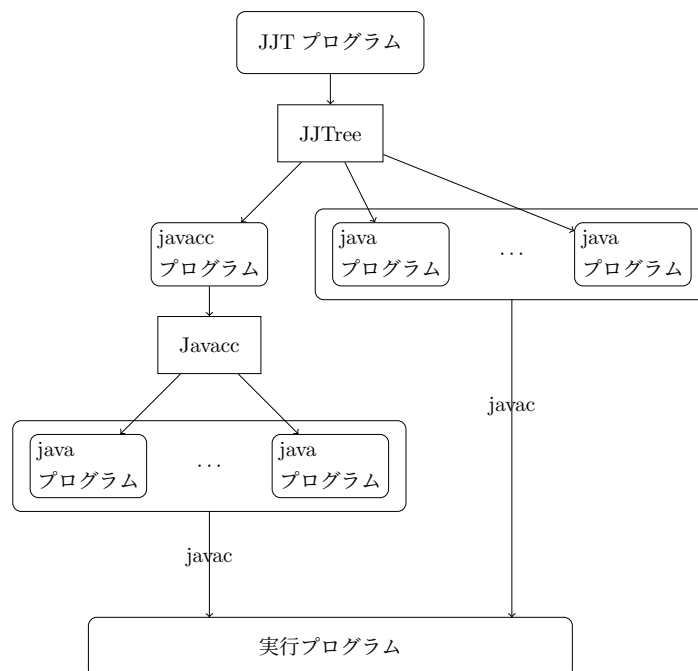


図 4.3: jjtree-config

4.2.6 例

(本内容の javacc コードは、早乙女健治 (著) : Javacc : コンパイラ・コンパイラ for Java (テクノプレス刊 2003) からの引用によるものです.)

以下の拡張 CF 文法で定義された四則演算式を構文解析する.

$$\begin{aligned}\langle \text{AddExpr} \rangle &::= \langle \text{MulExpr} \rangle ((+|-) \langle \text{MulExpr} \rangle) * \\ \langle \text{MulExpr} \rangle &::= \langle \text{PrimExpr} \rangle ((*|/) \langle \text{PrimExpr} \rangle) * \\ \langle \text{PrimExpr} \rangle &::= '(' \langle \text{AddExpr} \rangle ')' | \text{'Literal'}\end{aligned}$$

```
/** simple calculator */
options {
    STATIC=false;
    MULTI=true;
}
PARSER_BEGIN(Calc)
public class Calc {
    public static void main(String args[]) throws ParseException {
        while (true) {
            Calc parser = new Calc(System.in);
            switch (parser.CalcUnit()) {
                case -1:
                    return;
                case 0:
                    continue;
                default:
                    //      ((SimpleNode)parser.jjttree.rootNode()).dump("");
                    parser.jjttree.rootNode().calculate();
            }
        }
    }
}
PARSER_END(Calc)
SKIP:
{
    " " | "\r"
}

TOKEN:
{
    < ADDOP: "+" >
```

```

| < SUBOP: "-" >
| < MULOP: "*" >
| < DIVOP: "/" >
| < LP: "(" >
| < RP: ")" >
| < LITERAL: (["0"-"9"])* >
| < NL: "\n" >
}

int CalcUnit() :
{
    ( AddExpr() <NL>
      { return 1; }
    | <NL>
      { return 0; }
    | <EOF>
      { return -1; }
    )
}

void AddExpr() #void :
{
    {
        MulExpr()
        (
            <ADDOP> MulExpr() #AddNode(2)
        | <SUBOP> MulExpr() #SubNode(2)
        )*
    }
}

void MulExpr() #void :
{
    {
        PrimExpr()
        (
            <MULOP> PrimExpr() #MulNode(2)
        | <DIVOP> PrimExpr() #DivNode(2)
        )*
    }
}

void PrimExpr() #void :
{

```

```

{
    Literal()
    | <LP> AddExpr() <RP>
}
void Literal() :
{ Token t; }
{
    t=<LITERAL>
    { jjtThis.litValue = Integer.parseInt(t.image); }
}

```

AddExpr() 内の, #AddNode(2), #SubNode(2) MulExpr() 内の, #MulNode(2), #DivNode(2) はそれぞれ, 四則演算子が2項演算なので, 2つノードが生成されれば, 新たにノードを生成する効果を持つ。

4.3 上昇型構文解析

下降型構文解析は単純で効率的な構文解析手法ではあるが, CF 文法が実用的には LL(1) 文法である必要がある, LL(1) 言語に用いることのできる CF 文法の生成規則の適用と意図する構文木の対応が必ずしも明確ではないなどの特徴があるため, 比較的シンプルで構文と意味の間に柔軟性がある言語の解析に向いている。下降型では, 入力系列を読み取ることなく, 構文木の根の方向から適用する生成規則を限られた先読み文字によって決定することができなければならない。

これに対して, 生成規則の右辺を入力および入力から導出の逆動作 (還元) で得られた文形式に対してマッチングすることで, 左辺に置き換える還元を行って, 開始記号まで還元することによって, 逆方向から導出を構成する構文解析手法がある。これは, 構文木の葉の方向から根を構成するので**上昇型構文解析**と呼ばれる。ここでは, 上昇型構文解析の計算機構であるプッシュダウン変換機械 (pushdown transducer) を示し, この上での上昇型構文解析として LR 構文解析を示す。一般に用いられる LR(1) 構文解析は LL(1) 構文解析よりも適用できる CF 文法の範囲が大きい。Yacc などの構文解析プログラム生成ツールが対象としている LALR(1) 構文解析は LL(1) 構文解析とは適用できる CF 文法の範囲は比較できないが, 左括りだしや左再帰性の除去などが必要なく, 構文木を直接的に得られるためよく用いられる。LR 構文解析として, SLR(1) 構文解析手法を示したのち, その拡張として LR(1) 構文解析手法を示す。

4.3.1 プッシュダウン変換機械

プッシュダウン変換機械 (pushdown transducer, PDT) は, プッシュダウンオートマトンに出力テープを追加した計算モデルである。プッシュダウンオートマトン (PDA) はスタックへのプッシュ, ポップ動作によってスタック記号を容量制限のない記憶に格納することができる。プログラミング言語は文脈自由文法で定義されることから, その受理にはプッシュダウンオートマトンを用いる必要がある。さらに構文解析の際にはどの生成規則が使われたかを出力する必要があるので, 出力

テープを持つ PDT を用いる。下降型構文解析においては、入れ子構造をもつ手続き呼び出しがスタックの働きをしている。上昇型構文解析では、構文解析表に従って動作する PDT を構成する。

定義 4.8. PDT を $T = \langle \Gamma, \Sigma, O, \Delta, I_0, F \rangle$ として定義する。ここで

- Γ スタック記号の有限集合
- Σ 入力記号の有限集合
- O 出力記号の有限集合
- Δ 遷移関数 $\Gamma^+ \times \Sigma \rightarrow \Gamma^* \times O^* \times \{0, 1\}$
- $I_0 \in \Gamma$ 初期スタック記号
- $F \subseteq \Gamma \cup \{\varepsilon\}$ 最終スタック記号の集合

Σ には特別の記号 $\#$ を含む。 $\#$ は入力テープの右端を示す希望である。 (s, i, o) の 3 項組を時点表示 (configuration) という。スタック $s \in \Gamma^*$ はスタック記号の有限系列、入力テープ $I \in \Sigma^*$ 、出力テープ $o \in O^*$ とする。 s は左側がスタックの底、右側がスタックトップを表す。遷移関数 $\Delta : \Gamma^+ \times \Sigma \rightarrow \Gamma^* \times O^* \times \{0, 1\}$ によって時点表示を更新する。

- $\Delta(u, a) = (w, \pi, 1)$ のとき、遷移を $u \xrightarrow{a} (w, \pi)$ と書き、PDT T の時点表示の更新を $(vu, ax, o) \vdash (vw, x, o\pi)$ で表す。
- $\Delta(u, a) = (w, \pi, 0)$ のとき、遷移を $u \xrightarrow{(a)} (w, \pi)$ と書き、PDT T の時点表示の更新を $(vu, ax, o) \vdash (vw, ax, o\pi)$ で表す。

$u \xrightarrow{a} (w, \pi)$ を時点表示 (vu, ax, o) に適用するときには、 T のスタックの上部 u を w に置き換え、出力テープに π を追加して、 $o\pi$ とする。 $u \xrightarrow{(a)} (w, \pi)$ を時点表示 (vu, ax, o) に適用するときには、スタックの上部 u を v に置き換えて、出力テープに π を追加して $o\pi$ とする。このとき、入力 a は読み込まない。遷移を行うためには入力テープの先頭は a である必要があるが、入力テープのポインタは動かさない。この動作を先読みといい、 a を先読み記号という。

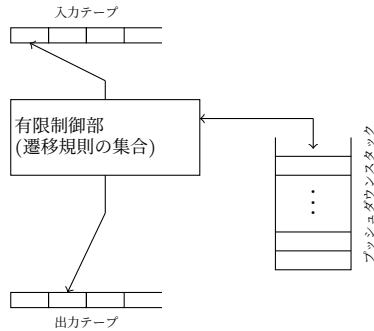


図 4.4: プッシュダウン変換機械 (PDT)

4.3.2 LR 構文解析

CF 文法の上昇型構文解析器 (bottom-up parser) は、入力として $x\# \in \Sigma^*$ を受取り、 $x\#$ から導出の逆関係 $\Leftarrow$ を用いて、入力の左側から還元 (reduction) により開始記号 S を得る。

$$w = \alpha_n \Leftarrow \alpha_{n-1} \Leftarrow \cdots \Leftarrow \alpha_1 \Leftarrow \alpha_0 = S$$

ここで、 $\beta\alpha\gamma \Leftarrow \beta A\gamma$ となる生成規則 $A \rightarrow \alpha$ が存在する。 α_0 においては、 $\beta = \varepsilon$ である。 上記の還元系列は、逆方向から見ると最右導出になっている。 例えば $\alpha_1 = AB \Leftarrow S$ であるとき、 $\alpha_2 = A\beta \Leftarrow AB$ である。 $A \rightarrow \alpha$ による還元が $\alpha_{i+1} = \beta\alpha\gamma \Leftarrow \beta A\gamma = \alpha_i$ であるとき、 α を α_{i+1} のハンドル (handle) という。 α_i から決定的にハンドルを見つけて還元することが効率的な構文解析を構成するために必要である。 α_i におけるハンドルが α である場合、 $\alpha_i = \beta\alpha\gamma$, $\beta \in \Sigma^*$ であり、 β は入力 w の接頭語になっている。

上昇型構文解析器を構成するとき、CF 文法 $G = \langle N, \Sigma, P, S \rangle$ から**拡大文法** (augmented grammar) $G' = \langle N \cup \{S'\}, \Sigma \cup \{\#\}, P \cup \{S' \rightarrow S\# \}, S' \rangle$ を構成する。 ここで、 S' は新たな非終端記号であり、 $\#$ は終端記号である。 拡大文法における上昇型構文解析では $S' \rightarrow S\#$ による還元か、ハンドルが見つからない場合に終了する。 前者の場合、最右導出が構成できて $w \in L(G)$ が示され、後者の場合は、 $w \notin L(G)$ であり、構文エラーである。

例 4.4. 文法 $\langle \{i, n, +, *, (,)\}, \{E\}, P, E \rangle$ に対する拡大文法は $\langle \{i, n, +, *, \#, (,)\}, \{E, S'\}, P \cup \{S' \rightarrow E\# \}, S' \rangle$ となる。 ここで、

$$P = \left\{ \begin{array}{ll} 0: S' \rightarrow E\# & 3: E \rightarrow (E) \\ 1: E \rightarrow E + E & 4: E \rightarrow i \\ 2: E \rightarrow E * E & 5: E \rightarrow n \end{array} \right\}$$

入力系列を $(i+n)+i*i\#$ とする。

$$\begin{aligned} S' &\Rightarrow_{rm} E\# \\ &\Rightarrow_{rm} E + E\# \\ &\Rightarrow_{rm} E + E * E\# \\ &\Rightarrow_{rm} E + E * i\# \\ &\Rightarrow_{rm} E + i * i\# \\ &\Rightarrow_{rm} (E) + i * i\# \\ &\Rightarrow_{rm} (E + E) + i * i\# \\ &\Rightarrow_{rm} (E + n) + i * i\# \\ &\Rightarrow_{rm} (i + n) + i * i\# \end{aligned}$$

これを還元によって受理する PDT の動作は以下ようになる。

$(I_0, (i+n)+i*i\#, \varepsilon)$		
$\vdash$	$(I_0 (,$	$i+n)+i*i\#, \varepsilon)$
$\vdash$	$(I_0 (i,$	$+n)+i*i\#, \varepsilon)$
$\vdash$	$(I_0 (E,$	$+n)+i*i\#, 4)$
$\vdash$	$(I_0 (E+,$	$n)+i*i\#, 4)$
$\vdash$	$(I_0 (E+n,$	$) + i*i\#, 4)$
$\vdash$	$(I_0 (E+E,$	$) + i*i\#, 45)$
$\vdash$	$(I_0 (E+E),$	$+i*i\#, 45)$
$\vdash$	$(I_0 (E),$	$+i*i\#, 451)$
$\vdash$	$(I_0 E,$	$+i*i\#, 4513)$
$\vdash$	$(I_0 E+,$	$i*i\#, 4513)$
$\vdash$	$(I_0 E+i,$	$*i\#, 4513)$
$\vdash$	$(I_0 E+E,$	$*i\#, 45134)$
$\vdash$	$(I_0 E+E*,$	$i\#, 45134)$
$\vdash$	$(I_0 E+E*i,$	$\#, 45134)$
$\vdash$	$(I_0 E+E*E,$	$\#, 451344)$
$\vdash$	$(I_0 E+E,$	$\#, 4513442)$
$\vdash$	$(I_0 E,$	$\#, 45134421)$
$\vdash$	$(I_0 E\#,$	$\varepsilon, 45134421)$
$\vdash$	$(I_0 S',$	$\varepsilon, 451344210)$

定義 4.9. 最右導出 $S' \Rightarrow_{rm}^* \alpha A w \Rightarrow_{rm} \alpha \beta w$ (ここで $w \in \Sigma\{\#\}$) において、 αA の接頭語および $\alpha \beta$ の接頭語を G' の **可延頭部** (*viable prefix*) という。

可延頭部は、上昇型構文解析において構文解析が進んでいる状態を示している。

SLR(1) 構文解析

定義 4.10. CF 文法 $G = \langle N, \Sigma, P, S \rangle$ とする。 $A \rightarrow \alpha \in P$ ($\alpha \in (N \cup \Sigma)^*$) に対して、 α の i 番目の記号を $\alpha(i) \in N \cup \Sigma$ で表す。また、 α の接頭語で長さが $k (\geq 0)$ の系列を $\alpha(: k)$ 、 α の接尾語で長さ $\ell (\geq 0)$ の系列を $\alpha(\ell :)$ で表すこととする。 $k = 0, \ell = 0$ のときはそれぞれ ε とする。各生成規則 $A \rightarrow \alpha$ に対する $LR(0)$ 項を以下のような形を持つ項として定義する。

$$[A \rightarrow \alpha(: k) \bullet \alpha(\ell :)]$$

ここで、 $|\alpha| = k + \ell$ である。

G から得られる $LR(0)$ 項の集合を $LR(0)_G$ と書く。

$A \rightarrow \alpha \in P$ に対して、 $|\alpha| = m$ のとき、 $m + 1$ 個の $LR(0)$ 項が存在する。

例 4.5. 文法 $G_{Arith} = \langle \{E, T, F\}, \{i, +, *, (,)\}, P, E \rangle$ とする. ここで,

$$P = \left\{ \begin{array}{l} E \rightarrow E+T \mid T, \\ T \rightarrow T*F \mid F, \\ F \rightarrow i \mid (E) \end{array} \right\} \text{ とする. } P \text{ から得られる } LR(0) \text{ は以下のようなになる.}$$

$$\begin{array}{llll} [E \rightarrow \bullet E+T] & [E \rightarrow E \bullet +T] & [E \rightarrow E+\bullet T] & [E \rightarrow E+T \bullet] \\ [E \rightarrow \bullet T] & [E \rightarrow T \bullet] & & \\ [T \rightarrow \bullet T*F] & [T \rightarrow T \bullet *F] & [T \rightarrow T*\bullet F] & [T \rightarrow T*F \bullet] \\ [T \rightarrow \bullet F] & [T \rightarrow F \bullet] & & \\ [F \rightarrow \bullet (E)] & [F \rightarrow (\bullet E)] & [F \rightarrow (E \bullet)] & [F \rightarrow (E) \bullet] \\ [F \rightarrow \bullet i] & [F \rightarrow i \bullet] & & \end{array}$$

定義 4.11. 拡大文法の開始記号を S' とする. $A \rightarrow \alpha_1 \alpha_2$ を生成規則に含む. $S' \Rightarrow_{rm}^* \beta A w \Rightarrow_{rm} \beta \alpha_1 \alpha_2 w$ となる導出が存在するとき, $[A \rightarrow \alpha_1 \bullet \alpha_2]$ を可延頭部 $\beta \alpha_1$ に対する正準 $LR(0)$ 項と呼ぶ. 可延頭部 $\beta \alpha_1$ に対する正準 $LR(0)$ 項の集合を $I_0(\beta \alpha_1)$ と書く.

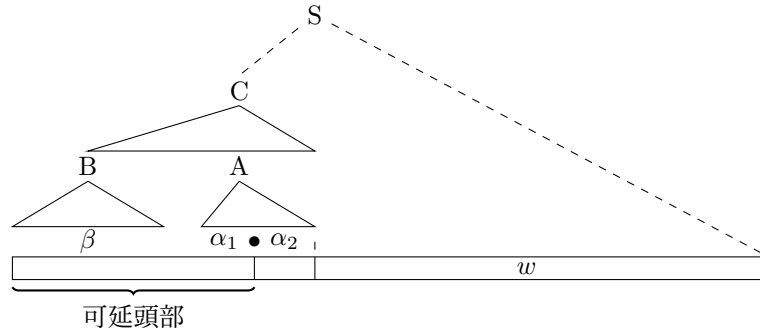


図 4.5: 導出と可延頭部

LR 構文解析は, 生成規則の右辺のマッチングによって構文木を構成する. $LR(0)$ 項は $\bullet$ までその生成規則の右辺にマッチングしている状態を示す (図 4.5). 可延頭部 $\gamma \in (N \cup \Sigma)^*$ に対する正準 $LR(0)$ 項は, 以下のような遷移系を定義することによって計算することができる.

定義 4.12. CF 文法 G に対して $LR(0)_G$ 上に以下の遷移関係 $\xrightarrow{M}$ を定義する. ($M \in N \cup \Sigma \cup \{\varepsilon\}$) $A \rightarrow \alpha \in P$ において

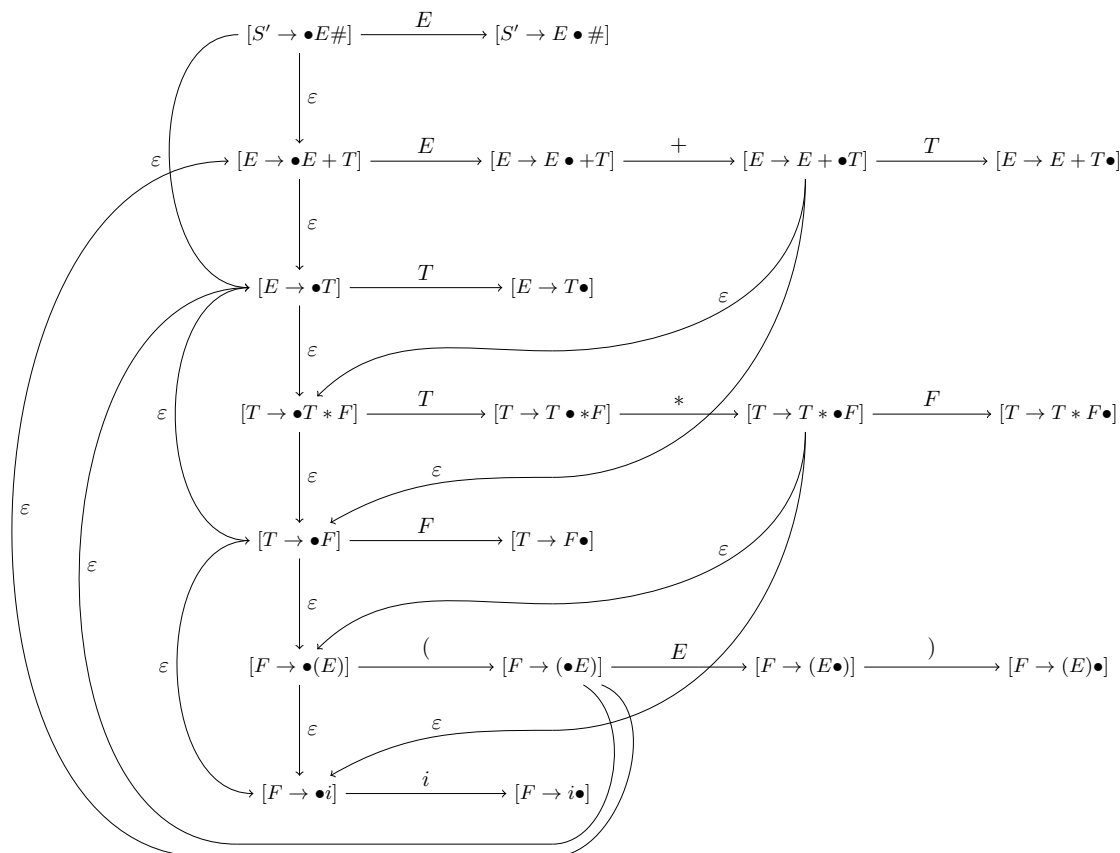
$$a \in \Sigma \text{ のとき } [A \rightarrow \beta \bullet a \beta'] \xrightarrow{a} [A \rightarrow \beta a \bullet \beta']$$

$$X \in N \text{ のとき } [A \rightarrow \beta \bullet X \beta'] \xrightarrow{X} [A \rightarrow \beta X \bullet \beta'] \text{ および}$$

$$X \rightarrow \gamma \in P \text{ に対して } [A \rightarrow \beta \bullet X \beta'] \xrightarrow{\varepsilon} [X \rightarrow \bullet \gamma]$$

$\theta \in LR(0)_G$ に対して $\text{act}(\theta) = \{M \mid \theta \xrightarrow{\varepsilon}^* \xrightarrow{M} \theta', M \neq \varepsilon \text{ となる } \theta' \text{ が存在する} \}$
 $I \subseteq LR(0)_G$ に対して $\text{act}(I) = \cup_{\theta \in I} \text{act}(\theta)$ とする.

例 4.6. 例 4.5 の文法 G_{Arith} に対する遷移は図 4.6 のようになる.

図 4.6: G_{Arith} に対する LR(0) 遷移

LR(0) 項を状態として、 $\bullet$ が次のシンボルに移動する遷移系によって、可延頭部が計算できる。拡大文法を考えると $[S' \rightarrow \bullet S\#]$ から $[A \rightarrow \alpha_1 \bullet \alpha_2]$ への遷移

$$[S' \rightarrow \bullet S\#] \xrightarrow{M_1} \dots \xrightarrow{M_n} [A \rightarrow \alpha_1 \bullet \alpha_2]$$

が存在するとき、連接 $M_1 M_2 \dots M_n$ が可延頭部となる。(M_i には ϵ も含まれる。)

定義 4.13. I を LR(0) 項の集合とする。 I と $\alpha \in (N \cup \Sigma)^*$ に対して Goto 関数を以下のように定義する。

$$\text{Goto}(I, \alpha) = \begin{cases} \{\theta' \mid \theta \xrightarrow{\epsilon}^* \theta' \text{ となる } \theta \in I \text{ が存在する} \} & \alpha = \epsilon \text{ のとき} \\ \text{Goto}(\{\theta' \mid \theta \xrightarrow{M} \theta' \text{ となる } \theta \in \text{Goto}(I, \beta) \text{ が存在する} \}) & \alpha = \beta M, M \neq \epsilon \text{ のとき} \end{cases}$$

可延頭部に対する正準 LR(0) 項の集合は、LR(0) 遷移系を NFA とみなして、部分集合構成法を用いて決定的な遷移をもつ遷移系に変換することによって求めることができる。

アルゴリズム 4.4. 以下の手順に沿って構成される遷移系 $\Theta_0 = (S_0, \mathcal{E}_0)$ を正準 LR(0) 遷移系とよぶ。 S_0 は Θ_0 の状態、 $\mathcal{E}_0$ は Θ_0 の辺である。

ステップ 1: $I_0 = \text{Goto}([S' \rightarrow \bullet S\#], \varepsilon)$ とする.

ステップ 2: $\Theta_0 \leftarrow (\{I_0\}, \emptyset); T \leftarrow \{I_0\}$

ステップ 3: $T = \emptyset$ ならば終了する. Θ_0 に正準 $LR(0)$ 遷移系が得られる.

ステップ 4: $I \in T$ を選び, T から削除する.

ステップ 5: すべての $M \in \text{act}(I)$ に対して, $I' = \{\theta' \mid \theta \in I, \theta \xrightarrow{\varepsilon^*} \xrightarrow{M} \xrightarrow{\varepsilon^*} \theta'\}$ を構成する.

$I' \notin S$ ならば, T に I' を S に I' を追加し, $\mathcal{E}$ に $I \xrightarrow{M} I'$ を追加する.

$I' \in S$ ならば, $\mathcal{E}$ に $I \xrightarrow{M} I'$ を追加する.

ステップ 6: ステップ 3 へもどる.

正準 $LR(0)$ 遷移系は, 正準 $LR(0)$ オートマトンと言われることもある.

命題 4.1. θ が可延頭部 α に対する正準 $LR(0)$ 項であるとき, かつそのときに限り $\theta \in \text{Goto}(I_0, \alpha)$ である.

例 4.7. G_{Arith} の正準 $LR(0)$ 遷移系は図 4.7 のようになる.

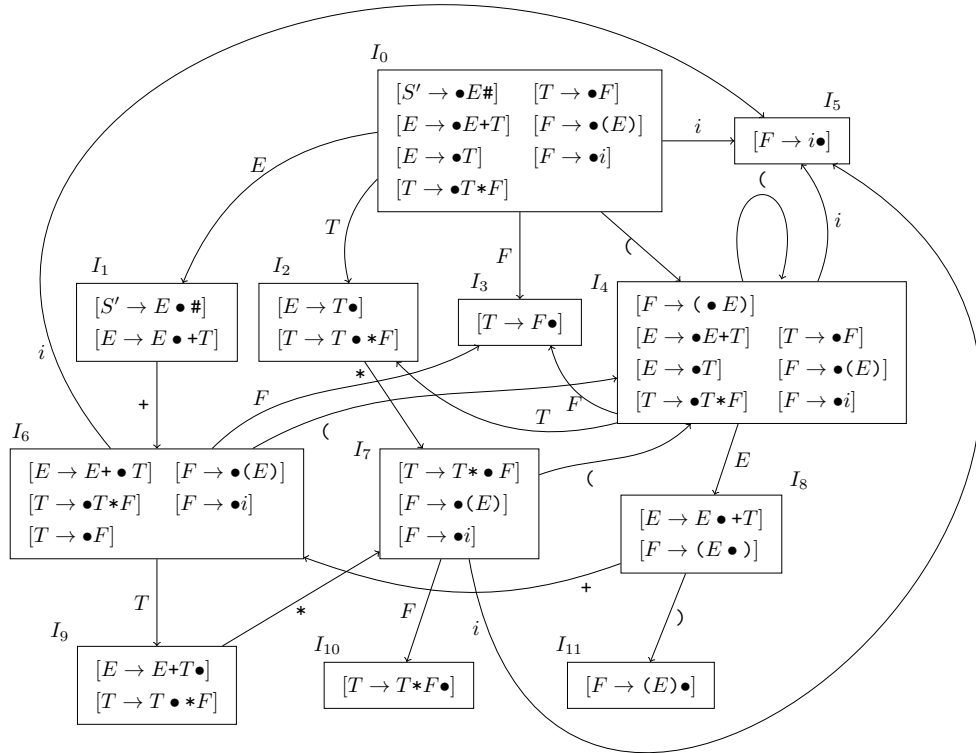


図 4.7: G_{Arith} の $LR(0)$ 遷移系

LR(0) 構文解析 正準 LR(0) 遷移系を PDT 上で動作させることによって LR(0) 構文解析を行う。PDT のスタック上には常に次の形で記号を書き込む。ここで、 I_i は LR(0) 遷移系の状態であり、 M_i は $N \cup \Sigma$ の要素である。

$$I_0 M_1 I_1 M_2 \cdots M_n I_n$$

スタックトップから $M_{i+1}, \dots, M_n$ を抽出して、生成規則の右辺とマッチさせる。マッチングが成功している場合、LR(0) 項は、 $[A \rightarrow M_{i+1} \cdots M_n \bullet]$ の形の LR(0) 項によって還元され、スタックの内容を A でおきかえる。これは A の遷移が可能になったことを示すので、 M_1 の前にある状態から A の遷移で移動可能な状態 I をスタックにプッシュし、使われた生成規則の番号を出力テープに出力する (*). これに対して、 $[A \rightarrow \alpha \bullet]$ という状態に達していない場合は、 M をスタックにプッシュして、移動した先の状態 I_i をプッシュする (†). (*) の振舞いを還元動作、(†) の振舞いをシフト動作と呼ぶ。

還元 スタックが $I_0 X_1 I_1 \cdots X_n I_n$ であり、 $[A \rightarrow X_{i+1} X_{i+2} \cdots X_n \bullet] \in I_n$ となる場合、スタックトップ (系列の右側) から $X_{i+1} I_{i+1} \cdots X_n I_n$ をポップして A をプッシュして、 $I_0 X_1 \cdots I_i A I'_{i+1}$ とする。ここで、 $I'_{i+1} = \text{Goto}(I_i, A)$ 、 $A \rightarrow X_{i+1} \cdots X_n$ の生成規則を r とする。

$$\begin{aligned} & (I_0 X_1 I_1 \cdots I_i X_{i+1} I_{i+1} \cdots X_n I_n, aw, \pi) \\ \vdash & (I_0 X_1 I_1 \cdots I_i A I'_{i+1}, aw, \pi r) \end{aligned}$$

シフト $a \in \Sigma$, $[A \rightarrow X_{i+1} \cdots X_n \bullet a X_{n+1} \cdots X_m] \in I_n$ であるとき、スタックに $a I_{n+1}$ ($I_{n+1} = \text{Goto}(I_n, a)$) をプッシュする。

$$\begin{aligned} & (I_0 X_1 I_1 \cdots X_n I_n, aw, \pi) \\ \vdash & (I_0 X_1 I_1 \cdots X_n I_n a I_{n+1}, w, \pi) \end{aligned}$$

受理 $[S' \rightarrow X\# \in I_n]$ かつ入力系列が $\#$ であるとき、受理する。

$$\begin{aligned} & (I_0 S I_1, \#, \pi) \\ \vdash & (acc, \varepsilon, \pi 0) \end{aligned}$$

受理動作は、規則 $0(S' \rightarrow S\#)$ による還元動作である。この場合、スタックに特別な記号 acc を書き込む。

エラー 還元、シフト、受理のいずれもが不可能である場合、構文エラーとなり停止する。

I_0 のスタックから開始して、 $I_0 X_1 I_1 \cdots X_i I_i$ となっている場合、 $I_i = \text{Goto}(I_0, X_1 \cdots X_i)$ であり、 $\theta \in I_i$ は、可延頭部 $X_1 \cdots X_i$ に対する LR(0) 項である。同じ可延頭部による正準 LR(0) 項の集合の全体を**正準 LR(0) 集成** (canonical collection of LR(0) items) とよぶ。

これらの動作は以下のような PDT で実現される。

アルゴリズム 4.5.

ステップ 1: 正準 LR(0) 遷移系 $(S, \mathcal{E})$ を構成し、LR(0) 項 $[S' \rightarrow \bullet S\#]$ を含む S の要素を初期状態 I_0 とする。

ステップ 2: $[A \rightarrow X_1 \cdots X_n \bullet] \in I \in \mathcal{S}$, $a \in \Sigma$ に対して次の遷移規則を追加する.

$$(IX_1I_1 \cdots X_nI_n, a) \xrightarrow{(a)} (A, r)$$

ここで $I \xrightarrow{X_1} I_1, I_i \xrightarrow{X_i} I_{i+1} \in \mathcal{E}$ であり, r は $A \rightarrow X_1 \cdots X_n$ の生成規則の番号である.

ステップ 3: $I \xrightarrow{M} I' \in \mathcal{E}$ に対して $\theta = [A \rightarrow X_1 \cdots X_i \bullet aX_{i+1} \cdots X_n] \in I$, $\theta' = [A \rightarrow X_1 \cdots X_i a \bullet X_{i+1} \cdots X_n] \in I'$ のとき次の遷移規則を追加する. ($a \in \Sigma$)

$$(I_0X_1I_1 \cdots X_iI, a) \xrightarrow{a} (I_0X_1I_1 \cdots X_iIaI', \varepsilon)$$

ここで $I_i \xrightarrow{X_{i+1}} I_{i+1} \in \mathcal{E}$ である.

ステップ 4: $[S' \rightarrow S \bullet \#] \in I \in \mathcal{S}$ に対して次の遷移規則を追加する.

$$(I'SI, \#) \xrightarrow{\#} (acc, 0)$$

この PDT を $PDT_{LR0(G)}$ と書くことにする.

命題 4.2. CF 文法 G に対して, 拡大文法を G' とする. $w \in L(G)$ であるとき, かつ, そのときに限り, $PDT_{LR0(G')}$ は $w\#$ を受理する.

定義 4.14. CF 文法 G の拡大文法を G' とする. CF 文法 G による $w \in \Sigma^*$ に対する $LR(0)$ 構文解析は, $PDT_{LR0(G')}$ において, $(I, w\#, \varepsilon)$ からの $\vdash$ による計算である. $PDT_{LR0(G')}$ の遷移関数 Δ において, $\Delta(u, a)$ の数が任意の u, a に対してたかだか 1 であるとき, **LR(0) 構文解析可能**である. $LR(0)$ 構文解析可能な CF 文法を **LR(0) 文法**という.

LR(0) 文法はそのままではあまり実用的ではない. G_{Arith} の LR(0) 遷移系において, I_2 の状態は $[E \rightarrow T \bullet]$ を含み, $[E \rightarrow T \bullet * F]$ も含むことから, PDT は入力記号 $*$ に対して, 還元とシフトに相当する遷移が可能である. (すなわち G_{Arith} は LR(0) 文法ではない.)

$(u, a) \xrightarrow{(a)} (AI, \pi)$ となる (AI, π) が複数ある場合, **Reduce/Reduce コンフリクト**, $(u, a) \xrightarrow{(a)} (AI, \pi)$ と $(u, a) \xrightarrow{a} (uaI, a)$ が両方存在する場合 **Shift/Reduce コンフリクト**があるという.

先読み文字の機能を導入すれば, I_2 において $[E \rightarrow T \bullet]$ からの還元動作で受理に至る可能性があるのは, $\text{Follow}_1(E)$ に先読み文字が含まれている場合である. 先読み文字が $*$ であれば, スタックに $+I_7$ を追加するシフト動作を行う. G_{Arith} において, $\text{Follow}_1(E) = \{\#, +, \cdot\}$ であるので, PDT の動作が定まる.

アルゴリズム 4.5 の PDT の構成を以下のように修正する.

ステップ 2': $[A \rightarrow X_1 \cdots X_n \bullet] \in I \in \mathcal{S}$ に対して $a \in \text{Follow}_1(A)$ に対して次の遷移規則を追加する.

$$(IX_1I_1 \cdots X_nI_n, a) \xrightarrow{(a)} (A, r)$$

ここで $I \xrightarrow{X_1} I_1, I_i \xrightarrow{X_i} I_{i+1} \in \mathcal{E}$ であり, r は $A \rightarrow X_1 \cdots X_n$ の生成規則の番号である.

この修正を適用して得られる PDT を $PDT_{SLR(1)}$ と書く.

定義 4.15. CF 文法 G の拡大文法を G' とする. CF 文法 G による $w \in \Sigma^*$ に対する $SLR(1)$ 構文解析は, $PDT_{SLR(1)(G')}$ において, $(I, w\#, \varepsilon)$ からの $\vdash$ による計算である. $PDT_{SLR(1)(G')}$ の遷移関数 Δ において, $\Delta(u, a)$ の数が任意の u, a に対してたかだか 1 であるとき, **SLR(1) 構文解析可能** である. $SLR(1)$ 構文解析可能な CF 文法を **SLR(1) 文法** という.

例 4.8. G_{Arith} に対する PDT は図 4.8 のように定義される.

Shift		Reduce	
		$I_0EI_1 \rightarrow (\#)$	(Acc, ε)
		$I_0EI_1 + I_6TI_9 \rightarrow (\#, +,)$	$(I_0EI_1, 1)$
		$I_0TI_2 \rightarrow (\#, +,)$	$(I_0EI_1, 2)$
		$I_0TI_2 * I_7FI_{10} \rightarrow (\#, *, +,)$	$(I_0TI_2, 3)$
$I_0 \rightarrow ($	$(I_0(I_4, \varepsilon))$	$I_0FI_3 \rightarrow (\#, *, +,)$	$(I_0TI_2, 4)$
$I_0 \rightarrow i$	(I_0iI_5, ε)	$I_0(I_4EI_8)I_{11} \rightarrow (\#, *, +,)$	$(I_0FI_3, 5)$
$I_1 \rightarrow +$	$(I_1 + I_6, \varepsilon)$	$I_0iI_5 \rightarrow (\#, *, +,)$	$(I_0FI_3, 6)$
$I_2 \rightarrow *$	$(I_2 * I_7, \varepsilon)$	$I_4EI_8 + I_6TI_9 \rightarrow (\#, +,)$	$(I_4EI_8, 1)$
$I_4 \rightarrow ($	$(I_4(I_4, \varepsilon))$	$I_4TI_2 \rightarrow (\#, +,)$	$(I_4EI_8, 2)$
$I_6 \rightarrow i$	(I_6iI_5, ε)	$I_4TI_2 * I_7FI_{10} \rightarrow (\#, *, +,)$	$(I_4TI_2, 3)$
$I_7 \rightarrow i$	(I_7iI_5, ε)	$I_4FI_3 \rightarrow (\#, *, +,)$	$(I_4TI_2, 4)$
$I_7 \rightarrow ($	$(I_7(I_4, \varepsilon))$	$I_4(I_4EI_8)I_{11} \rightarrow (\#, *, +,)$	$(I_4FI_3, 5)$
$I_8 \rightarrow +$	$(I_8 + I_6, \varepsilon)$	$I_4iI_5 \rightarrow (\#, *, +,)$	$(I_4FI_3, 6)$
$I_8 \rightarrow)$	$(I_8)I_{11}, \varepsilon)$	$I_6TI_9 * I_7FI_{10} \rightarrow (\#, *, +,)$	$(I_6TI_9, 3)$
$I_9 \rightarrow *$	$(I_9 * I_7, \varepsilon)$	$I_6FI_3 \rightarrow (\#, *, +,)$	$(I_6TI_9, 4)$
		$I_6(I_4EI_8)I_{11} \rightarrow (\#, *, +,)$	$(I_6FI_3, 5)$
		$I_6iI_5 \rightarrow (\#, *, +,)$	$(I_6FI_3, 6)$
		$I_7(I_4EI_8)I_{11} \rightarrow (\#, *, +,)$	$(I_7FI_{10}, 5)$
		$I_7iI_5 \rightarrow (\#, *, +,)$	$(I_7FI_{10}, 6)$

図 4.8: $PDT_{SLR(1)(G_{Arith})}$

SLR(1) 構文解析表 $PDT_{SLR(1)(G')}$ の遷移関数を構文解析表の形で書くことが多い. $SLR(1)$ 構文解析表では, $LR(0)$ 遷移系の状態と先読み文字による動作の対応 (action 表) と, 非終端記号 A を還元した後の PDT の振舞い (GOTO 表) を示す.

	<i>action</i>				<i>GOTO</i>		
	...	<i>a</i>	...	#	...	<i>A</i>	...
I_0							
$\vdots$							
I_n							

図 4.9: SLR(1) 構文解析表の形

action 表では、スタックトップにある LR(0) 遷移系の状態 I で先読み文字 $a \in \Sigma \cup \{\#\}$ となったとき、

$(uI, a) \xrightarrow{(a)} (A, \pi)$ が Δ で定義されていれば、 $r\pi$ を I の行の a の欄に追加する。

$(uI, a) \xrightarrow{a} (uIaI', \varepsilon)$ が Δ で定義されていれば、 sI' を I の行の a の欄に追加する。

$r\pi$ は還元動作、 sI はシフト動作を意味する。

GOTO 表では、 $(uI, a) \xrightarrow{(a)} (AI', \pi)$ が Δ で定義されていれば、 I' を I の行の A の欄に追加する。

構文解析表を構成して action 表にただか 1 つの動作しか記入されていなければ、SLR(1) 構文解析可能であり、その文法は SLR(1) 文法である。LR(0) 遷移系の構成から GOTO 表の欄にはただか 1 つの状態しか記入されない。

例 4.9. G_{Arith} に対する SLR(1) 構文解析表を図 4.10 に示す。

	action						goto		
	<i>i</i>	+	*	(	)	#	<i>E</i>	<i>T</i>	<i>F</i>
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

図 4.10: G_{Arith} の SLR(1) 構文解析表

LR(1) 構文解析

SLR(1) 構文解析は、上昇型構文解析として最も単純な解析手法である²。しかし、適用範囲はあまり大きくない。以下の CF 文法 $G_=_$ は、SLR(1) 構文解析可能ではない。

$$G_=_ = \langle \{S, E, T\}, \{=, +, i\}, P_=_ , S \rangle$$

開始記号を S' とした拡大文法 $G'_=_$ の生成規則は以下のようにになる。

$$P'_=_ = \left\{ \begin{array}{lll} 0: S' \rightarrow S\# & 1: S \rightarrow E=E & 2: S \rightarrow i \\ 3: E \rightarrow E+T & 4: E \rightarrow T & 5: T \rightarrow i \end{array} \right\}$$

$G'_=_$ の正準 LR(0) 遷移系と SLR(1) 構文解析表は以下のようにになる。

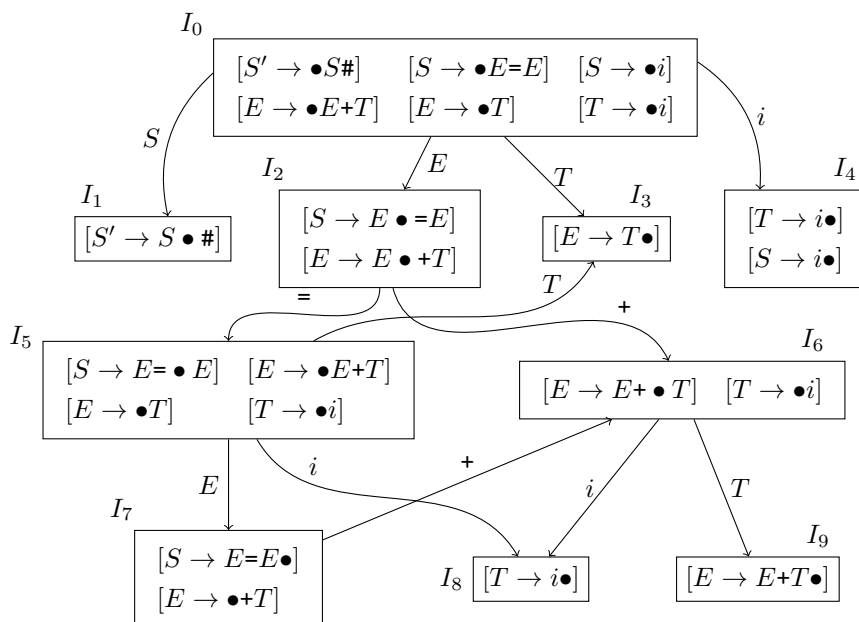


図 4.11: $G_=_$ の正準 LR(0) 遷移系

²SLR=Simple LR の意味

	action				Goto		
	i	$=$	$+$	$\#$	S	E	T
0	$s4$				1	2	3
1				acc			
2		$s5$	$s6$				
3		$r4$	$r4$	$r4$			
4		$r5$	$r5$	$r5$ $r2$			
5	$s8$					7	3
6	$s8$						9
7			$s6$	$r1$			
8		$r5$	$r5$	$r5$			
9		$r3$	$r3$	$r3$			

図 4.12: $G_=_$ の SRL(1) 構文解析表

状態 4 において reduce/reduce conflict が発生している． $\#$ が先読み文字のときに S に還元するか T に還元するかが定まらない．これを定めるためには，次に $=$ があるのかどうかでどちらの規則で還元するかを定める必要がある．

このため，マッチング状態に含まれる文脈情報を増やした LR(1) 構文解析を考える．LR(0) 項を構成する生成規則で還元されたときの次の文字を含めた LR(1) 項を考える．LR(1) 項に加える文字は，生成規則の左辺 A の $\text{Follow}_1(A)$ のみで十分である．他の LR(1) 項は受理計算には現れない．(すべてエラー状態である．)

定義 4.16. CF 文法 G およびその拡大 CF 文法を G' とする． S' 以外の非終端記号 A に対する生成規則 $A \rightarrow \alpha$ に対する LR(0) 項 $[A \rightarrow \beta_1 \bullet \beta_2]$ および $a \in \Sigma \cup \{\#\}$ の組 $[A \rightarrow \beta_1 \bullet \beta_2, a]$ および， $[S' \rightarrow \beta_1 \bullet \beta_2, \varepsilon]$ を **LR(1) 項** という．ここで， S' は G' ， S は G の開始記号である．

定義 4.17. 最右導出 $S' \Rightarrow_{rm}^* \alpha A w \Rightarrow \alpha \beta_1 \beta_2 w$ が存在し， $a = \text{First}_1(w)$ であるとき， $[A \rightarrow \beta_1 \bullet \beta_2, a]$ を可延頭部 $\alpha \beta_1$ に対する正準 LR(1) 項と呼ぶ．可延頭部 $\alpha \beta_1$ の集合を $I_1(\alpha \beta_1)$ と記述することにする．

定義 4.18. (1) $A \neq S'$ である LR(1) 項 $[A \rightarrow \beta_1 \bullet a \beta_2, a]$ ($a \in \Sigma$) に対して

$$[A \rightarrow \beta_1 \bullet a \beta_2, b] \xrightarrow{a} [A \rightarrow \beta_1 a \bullet \beta_2, b]$$

(2) $A \neq S'$ である LR(1) 項 $[A \rightarrow \beta_1 \bullet X \beta_2, a]$ ($X \in N$) に対して

$$\begin{aligned} [A \rightarrow \beta_1 \bullet X \beta_2, b] &\xrightarrow{X} [A \rightarrow \beta_1 X \bullet \beta_2, b] \\ [A \rightarrow \beta_1 \bullet X \beta_2, b] &\xrightarrow{\varepsilon} [X \rightarrow \bullet \gamma, c] \end{aligned}$$

ここで， $X \rightarrow \gamma \in P$ ， $c \in \text{Follow}_1(X)$

(3) $[S' \rightarrow \bullet S\#]$ に対して,

$$\begin{aligned} [S' \rightarrow \bullet S\#, \varepsilon] &\xrightarrow{S} [S' \rightarrow S \bullet \#, \varepsilon] \\ [S' \rightarrow \bullet S\#, \varepsilon] &\xrightarrow{\varepsilon} [S \rightarrow \bullet \gamma, \#] \end{aligned}$$

アルゴリズム 4.4 と同様に正準 LR(1) 遷移系を構成する.

アルゴリズム 4.6. 以下の手順に沿って構成される遷移系 $\Theta_1 = (S_1, \mathcal{E}_1)$ を**正準 LR(1) 遷移系**とよぶ. S_1 は Θ_1 の状態, $\mathcal{E}_1$ は Θ_1 の辺である.

ステップ 1: $I_1 = \text{Goto}([S' \rightarrow \bullet S\#, \varepsilon], \varepsilon)$ とする.

ステップ 2: $\Theta \leftarrow (\{I_1\}, \emptyset); T \leftarrow \{I_1\}$

ステップ 3: $T = \emptyset$ ならば終了する. Θ_1 に正準 LR(1) 遷移系が得られる.

ステップ 4: $I \in T$ を選び, T から削除する.

ステップ 5: すべての $M \in \text{act}(I)$ に対して, $I' = \{\theta' \mid \theta \in I, \theta \xrightarrow{\varepsilon^*} \xrightarrow{M} \xrightarrow{\varepsilon^*} \theta'\}$ を構成する.

$I' \notin S_1$ ならば, T に I' を S_1 に I' を追加し, $\mathcal{E}$ に $I \xrightarrow{M} I'$ を追加する.

$I' \in S$ ならば, $\mathcal{E}$ に $I \xrightarrow{M} I'$ を追加する.

ステップ 6: ステップ 3 へもどる.

LR(1) 項に対する Goto 関数は定義 4.13 と同様に定義できる.

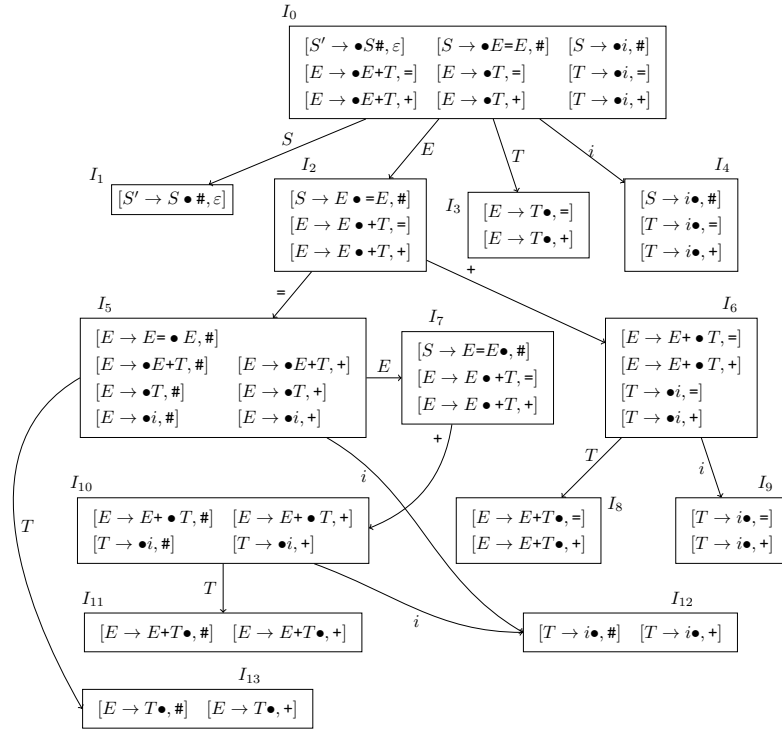
定義 4.19. I を LR(1) 項の集合とする. I と $\alpha \in (N \cup \Sigma)^*$ に対して Goto 関数を以下のように定義する.

$$\text{Goto}(I, \alpha) = \begin{cases} \{\theta' \mid \theta \xrightarrow{\varepsilon^*} \theta' \text{ となる } \theta \in I \text{ が存在する} \} & \alpha = \varepsilon \text{ のとき} \\ \text{Goto}(\{\theta' \mid \theta \xrightarrow{M} \theta' \text{ となる } \theta \in \text{Goto}(I, \beta) \text{ が存在する} \}) & \alpha = \beta M, M \neq \varepsilon \text{ のとき} \end{cases}$$

正準 LR(1) 遷移系 $(S_1, \mathcal{E}_1)$ において, $I \in S_1$ に対して, $I \xrightarrow{M} I' \in \mathcal{E}_1$ ならば, $\text{Goto}(I, M) = I'$ である.

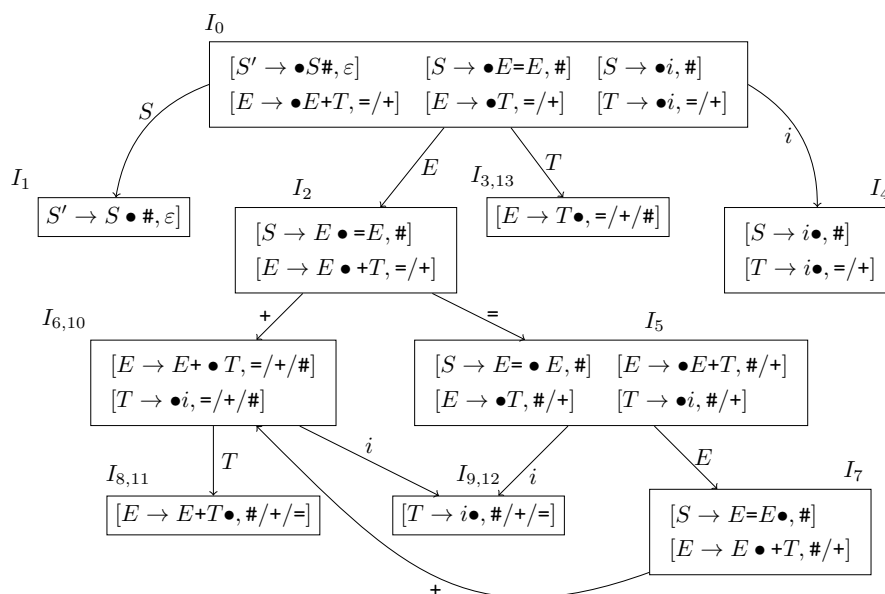
例 4.10. $G_{\leq}$ に対する正準 LR(1) 遷移系は図 4.13 のようになる.

さらに LR(1) 構文解析表は図 4.14 のようになる.

図 4.13: G_- の正準 LR(1) 遷移系

	i	$=$	$+$	$\#$	S	E	T
0	s4				1	2	3
1				acc			
2		s5	s6				
3		r4	r4				
4		r5	r5	r2			
5	s12					7	13
6	s9						8
7			s10	r1			
8		r3	r3				
9		r5	r5				
10	s12						11
11			r3	r3			
12			r5	r5			
13			r4	r4			

図 4.14: G_- の LR(1) 構文解析表

図 4.15: $G_ =$ の LALR(1) 遷移系

$G_ =$ の表エントリに操作の重複はなく、 $G_ =$ は $LR(1)$ 文法である。これは、正準 $LR(1)$ 遷移系の I_4 において、 $S \rightarrow i$ で還元するのは先読み文字が $\#$ の場合のみであり、それ以外の先読み文字の場合は $T \rightarrow i$ で還元することが $LR(1)$ 項によって区別できるためである。

LALR(1) 構文解析

正準 $LR(1)$ 遷移系は、 Σ の数が増えると状態数が増える。 $LR(1)$ 項に含まれる $LR(0)$ 項は共通の形をしている事があり、可延頭部までの構文解析の状態としては共通である。LALR(1) 構文解析は、含まれる $LR(0)$ 項が全く同じで先読み文字だけ違いがあるときに、構文解析としては同じ状態にあると考えて LR 遷移系の状態数を減らす構文解析である。

$G_ =$ の場合は、 I_6 と I_{10} 、 I_3 と I_{13} 、 I_9 と I_{12} は同じ状態にあるとして、図 4.15 のように LALR(1) 遷移系を構成する。

この遷移系に基づいて図 4.16 のように構文解析表を構成する。

図 4.16 のエントリにはただか 1 つの操作のみが書かれているため、 $G_ =$ は、LALR(1) 構文解析である。

正準 $LR(1)$ 遷移系から正準 LALR(1) 遷移系を構成する場合、最大で $LR(1)$ と同じ状態の遷移系を扱わなければならない。LALR(1) 遷移系は $LR(1)$ 遷移系を作成しなくても構成できる。LALR(1) 遷移系のより直接的な作成アルゴリズムについては参考書 [?] を参照のこと。

LALR(1) 構文解析は、可延頭部に着目する $LR(0)$ 項と同じくらいの状態数で、構文解析を行うことができる。ここで、先読み文字は状態に埋め込んでいることから $SLR(1)$ よりもより広い範囲の CF 文法に適用可能である。LALR(1) は構文解析生成プログラムの yacc が採用している構文解析である。 Σ の数が増えても、 $LR(1)$ 遷移系と比較して一定程度状態数を減らすことができるの

	action				goto		
	i	=	+	#	S	E	T
0	s4				1	2	(3,13)
1				acc			
2		s5	s(6,10)				
(3,13)		r4	r4	r4			
4		r5	r5	r2			
5						7	(3,13)
(6,10)	s(9,12)						(8,11)
7			s(6,10)	r1			
(8,11)		r3	r3	r3			
(9,12)		r5	r5	r5			

図 4.16: $G_{=}$ の LALR(1) 構文解析表

で、効率的な構文解析手法として非常によく用いられる。

4.3.3 構文解析手法の能力比較

図 4.17 に無曖昧な CF 文法の解析手法の比較についてしめす [?].

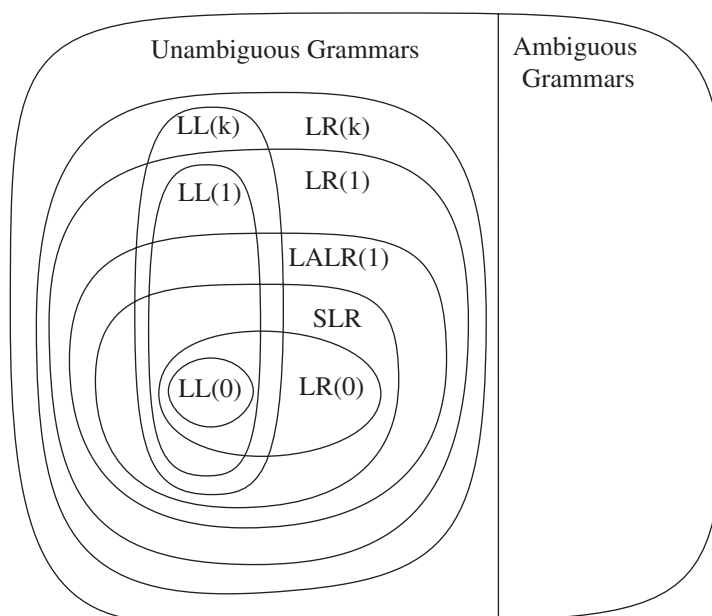


図 4.17: CF 文法の解析能力の比較

	i	n	$+$	$-$	$*$	$/$	$($	$)$	$\#$
i			$\gg$	$\gg$	$\gg$	$\gg$		$\gg$	$\gg$
n			$\gg$	$\gg$	$\gg$	$\gg$	$\ll$	$\gg$	$\gg$
$+$	$\ll$	$\ll$	$\gg$	$\gg$	$\ll$	$\ll$	$\ll$	$\gg$	$\gg$
$-$	$\ll$	$\ll$	$\gg$	$\gg$	$\ll$	$\ll$	$\ll$	$\gg$	$\gg$
$*$	$\ll$	$\ll$	$\gg$	$\gg$	$\gg$	$\gg$	$\ll$	$\gg$	$\gg$
$/$	$\ll$	$\ll$	$\gg$	$\gg$	$\gg$	$\gg$	$\ll$	$\gg$	$\gg$
$($	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$==$	
$)$			$\gg$	$\gg$	$\gg$	$\gg$		$\gg$	$\gg$
$\#$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	acc

表 4.1: 演算子順位表

4.3.4 演算子順位法

演算を持つ式を構文解析する方法として、演算子順位 (operator precedence) 法がある。この構文解析手法は演算子の優先度と結合の規則を直接的に構成できることから実現が容易で、効率的な構文解析が可能である。しかし、文法から生成できない文字列にも適用が可能であり、CF 文法との対応は不明確であり、LL, LR の構文解析のような CF 文法に基づいて柔軟に構文解析プログラムを生成することはできない。

$$G_{Exp} = \langle \{E\}, \{+, -, *, /, (,), i, n\}, P, E \rangle$$

$$P = \left\{ \begin{array}{l} 1: E \rightarrow E+E, \quad 2: E \rightarrow E-E, \quad 3: E \rightarrow E*E, \quad 4: E \rightarrow E/E, \\ 5: E \rightarrow (E), \quad 6: E \rightarrow i, \quad 7: E \rightarrow n \end{array} \right\}$$

G_{Exp} で定義される式の表現について考える。ここでの演算子は左結合であることから、左にある演算子が優先されることから、 $+\gg+, +\gg-, -\gg+, -\gg-$ および $*\gg*, *\gg/, /\gg*, /\gg/$ という順序をつける。ここでは通常の演算式のように $*, /$ は $+, -$ の演算よりも優先されることから、 $*\gg+, *\gg-, /\gg+, /\gg-$ という優先度を付加する。(については、中の式を先に解析するため (の優先度は低くなる。) との対応をとるために同じ優先度 $==$ を導入している。) は、括弧の中の解析を優先するため高い優先度を持つ。その他の演算子の間にも表 4.1 というような順序を定義する。

定義 4.20. $\alpha \in (N \cup \Sigma)^*$ に対して $\text{term}(\alpha)$ を α からすべての非終端記号を取り除いて得られる記号列とする。

演算子順位法による構文解析

演算子順位法で構文解析を行う PDT を定義する。

PDT のスタックを $a_1 \cdots a_n$ とする。ここでスタックの底 a_1 は $\#$ であるとする。スタックトップは a_n であり、入力でポインタのある次に読み込む記号を $b \in \Sigma \cup \{\#\}$ とする。

還元 $a_n \gg b$ ならば, $a_i \gg a_{i+1} == \dots == a_n$ となる $a_{i+1}, \dots, a_n$ をスタックから取り除く. $\text{term}(\alpha) = a_1 i + 1, \dots, a_n$ となる生成規則 $A \rightarrow \alpha$ の番号 o を出力テープに書き込む. Δ に以下の規則を含む.

$$(a_{i+1} \dots a_n, b) \xrightarrow{(b)} (\varepsilon, o)$$

シフト $a_n == b$ または $a_n \ll b$ ならば, 入力記号 b をスタックトップに格納する. Δ に以下の規則を含む.

$$(a_n, b) \xrightarrow{b} (a_n b, \varepsilon)$$

受理 $a_n = b = \#$ ならば, 受理状態となる. Δ に以下の規則を含む.

$$(\#, \#) \xrightarrow{\#} (acc, \varepsilon)$$

エラー それ以外の場合, 構文エラーとなる.

演算子順位法による構文解析を行うには, まず, スタックに $\#$ をセットして, PDT を動作させ acc に到達すれば, 構文としては正しい.

例 4.11. $i+i*i\#$ に対して表 4.1 の演算子順位表を使った構文解析は以下のようになる.

	$(\#, i+i*n\#, \varepsilon)$	
⊢	$(\#i, +i*n\#, \varepsilon)$	シフト $(\# \ll i)$
⊢	$(\#, +i*n\#, 6)$	$E \rightarrow i$ で還元 $(i \gg +)$
⊢	$(\#+, i*n\#, 6)$	シフト $(\# \ll +)$
⊢	$(\#+i, *n\#, 6)$	シフト $(+ \ll i)$
⊢	$(\#+, *n\#, 66)$	$E \rightarrow i$ で還元 $(i \gg *)$
⊢	$(\#+, n\#, 66)$	シフト $(+ \ll *)$
⊢	$(\#+*, n\#, 66)$	シフト $(* \ll i)$
⊢	$(\#+*, \#, 667)$	$E \rightarrow n$ で還元 $(n \gg \#)$
⊢	$(\#+, \#, 6673)$	$E \rightarrow E*E$ で還元 $(* \gg \#)$
⊢	$(\#, \#, 66731)$	$E \rightarrow E+E$ で還元 $(+ \gg \#)$
⊢	$(acc, 66731)$	受理

G_{Exp} のように生成規則が演算子を特定できるような場合は, 曖昧な文法に対しても比較的簡単に構文解析が可能である. 簡単な数式に対してはこのような条件が成立するため簡易的に構文解析プログラムを構成できる.

4.4 構文エラー表示と誤り回復

構文解析表に従って解析を進める際に対応する表 (下向き構文解析表, LR 構文解析表の action 表) に生成規則, 操作がない場合, 入力記号列は構文エラーを含む. コンパイラは構文エラーを含む入力を変換する必要はないが, 入力に対して何がエラーの原因かを表示して, 適切な修正を促すことが望ましい. このような誤り回復の方法としては, プログラムに記号の挿入, 削除を適用する

方法、あるいはパニックモードを用いる方法がある。プログラムに挿入、削除の修正を自動的に適用する方法は、適用が難しく時間がかかることに加えて、修正が必ずしもプログラマーが意図したものになるかわからないので、修正が決定できる必要があるため適用できる言語は限定的である。

パニックモードを用いる方法では、構文エラーに到達したときは、残りのプログラム中に分離記号 (デリミター) を見つけるまで入力を読み捨てて誤り回復を試みる。しかし、得られた結果が信頼できるものではなく、複数の構文エラーを一回の構文解析で指摘することができるようにすることが目的である。一般に、最初のエラーが発生した場所を詳しく表示することが重要である。

4.5 YACC による構文解析プログラム生成

YACC(yet another compiler compiler) は構文解析器 (パーザー) を CF 文法から自動生成するツールである³適用できる CF 文法のクラスは LALR(1) である。

yacc で記述するファイルは %% で区切られた 3 つの部分で記述される。

ヘッダー、データ構造、トークン定義

%%

CF 文法

%%

C プログラム (使用する関数の定義リスト)

yacc は lex とともに使われる。lex によって用いる単語を定義して字句解析を行い、その結果を yacc で生成されたトークン列を送って構文解析を行う。

以下に四則演算を含む数式の yacc 記述を示す。%token は lex で切り分けるトークンを定義している。% left は左結合の演算子のトークンを表し上の方のトークンの還元の優先度が低いものを上に記述する。ここでは、加減算の方が乗除算よりも優先度が低いのでこの順番で記述する。

CF 文法の生成規則は下にあるような形で記述する。生成規則の後ろに {...} で記述された動作をその生成規則が還元されたときに実行する。\$i は右辺の i 番目の記号の値を表し、\$\$ は左辺の非終端記号の値を示す。

C 言語プログラムの場所はこのファイル程度書いておけば変換時のエラーは発生しない。

```
%{
#include <stdio.h>
#include <stdlib.h>
#define YYDEBUG 1
int yylex();
int yyerror(char const *str);
```

³yacc は、AT&T がもともと権利をもつツールであるため、GNU では bison として実現されている。


```

extern int yylval;
%}
%token      NL
%token      NUMBER
%token      LPAR
%token      RPAR
%left      ADDOP SUBOP
%left      MULOP DIVOP
%%
s          : list
          ;
list       : /* empty */
          | list expr NL      {printf("%d\n", $2);}
          ;
expr       : expr ADDOP expr  {$$ = $1 + $3; }
          | expr SUBOP expr   {$$ = $1 - $3; }
          | expr MULOP expr   {$$ = $1 * $3; }
          | expr DIVOP expr   {$$ = $1 / $3; }
          | LPAR expr RPAR     {$$ = $2; }
          | NUMBER            {$$ = $1; }
          ;
%%
int yyerror(char const *str) {
    extern char *yytext;
    fprintf(stderr, "parser error near %s\n", yytext);
    return 0;
}

int main(void) {
    extern int yyparse(void);
    extern FILE *yyin;

    yyin = stdin;
    if (yyparse()) {
        fprintf(stderr, "Error Occurred!\n");
        exit(1);
    }
}

int yywrap() {
    return(1);
}

```

}

演算子による優先度を定義しない場合は、誤った構文木が得られるため計算結果が異なる可能性がある。(Web サイトに記載.) Shift/Reduce コンフリクトが発生した場合, Shift 動作を優先する.